UNIVERSITY
OF OSLO

Master's thesis

Beyond the Curse of Dimensionality? Deep Equilibrium Nets for the Analytic Climate Economy

A mapping and application of Deep Equilibrium Nets to the Analytic Climate Economy

Herman Vestvik Thieme

Economics

30 ECTS study points

Department of Economics

Faculty of Social Sciences

May 2024

Herman V. Thieme

Beyond the Curse of Dimensionality? Deep Equilibrium Nets for the Analytic Climate Economy

A mapping and application of Deep Equilibrium Nets to the Analytic Climate Economy

Supervised by

Christian P. Traeger

Abstract

This thesis introduces the application of Deep Equilibrium Nets (DEQNs) to the Analytic Climate Economy (ACE), aiming to lift the curse of dimensionality present in traditional solution methods. To the best of my knowledge, it is one of the first applications of this solution method to integrated assessment models (IAMs). It is also one of the first to contextualize this method in terms of the reinforcement learning literature. With that, this research is a step toward making it possible to relax restrictive assumptions typically necessary in existing models. In this revised version, with corrected equilibrium conditions and a scaling of the objective functions, the policy network recovers the analytical solution of ACE with the DICE 2016 and DICE 2023 production functions closely. The estimated consumption rate deviates from the analytical rate by less than 0.3 percentage points, the abatement rate by 0.1 to 0.2 percentage points on average, and the implied social cost of carbon deviates from its closed-form value by 0.2 to 0.6% on average. The results also show how sensitive the method is to the correctness of the equilibrium conditions: errors in the submitted version led the network to a wrong fixed point, which was mistakenly attributed to the normalization of inputs. The thesis suggests exploring more effective algorithms inspired by advances in reinforcement learning.

About this Revised Version

This is a revised version of the thesis. After submission, I found errors in the equilibrium conditions that make up the policy objective, both in the derivations and in the code used for the experiments:

- The envelope condition for capital omitted the direct effect of capital on utility, so the capital condition ℓ_0 did not hold at the optimum. In the code, the marginal product of capital was also missing from ℓ_0.
- The first-order conditions on the consumption rate (ℓ_3) and on emissions (ℓ_4) counted the effect of today's controls on the next-period value twice, once directly and once through the costate variables.
- The emissions condition omitted the multiplier on the upper bound $E_t \leq E_t^{\mathrm{BAU}}$ and the conversion from emissions in $\mathrm{GtCO_2}$ per year to the carbon stock in GtC per period.
- The time indices of the temperature condition (ℓ_2) were off by one period, and the code evaluated the Bellman equation (ℓ_6) with a wrongly transformed emission level.

At the analytical solution of ACE, the original conditions ℓ_0 and ℓ_3 are not zero, so the policy network was trained towards a wrong fixed point. Section (5), Appendix (A) and Algorithm (1) are corrected. The code was rewritten with the corrected conditions and a scaling of the objective functions (section 6.1.2), and all results in section (6) come from new training runs. The abstract, the discussion of the results and the conclusion are updated accordingly. The updated thesis, the corrected code and the new results are available on GitHub at `https://github.com/hermanvest/DNN-for-ACE` in the directory `revised/`. The repository is divided into this revised version and the original code, trained networks and results used for the submitted thesis, which remain unchanged in the repository root for reference.

Acknowledgements

My academic journey, all the way to my master's degree, has been an incredibly rewarding and enriching experience. I have faced several challenges that, at times, I did not think I would be able to overcome. Despite not considering myself exceptionally smart, I often find myself humorously resonating with the following quote:

> "Problems worthy of attack
> prove their worth by fighting back."
> - Piet Hein

However, I have been fortunate enough to be surrounded by many incredible people who supported me throughout this journey.

I would like to express my sincere gratitude to my supervisor, Christian Traeger, for providing me with an excellent research experience. Your kindness, patience, and guidance throughout the writing process have been invaluable and I am fortunate to have had the opportunity to work with you. Additionally, I would like to thank Danial Ali Akbari for introducing me to climate research and suggesting that I investigate DEQNs for my master's thesis.

A big thanks to my fellow students and research assistants in room 1221 for all the support and good vibes. Sharing this journey with you has been awesome, to say the least. A big thank you to Ulrik, who has an incredible work ethic and an even bigger heart. Lastly, I want to give a heartfelt thank you to Maren, for your patience, love, and endless support throughout the last few years.

Contents

1 Introduction **1**

2 Literature **3**
2.1 Analytical Integrated Assessment Models 3
2.2 Methods for Computing Economic Equilibrium 4

3 Preliminaries **8**
3.1 Discrete-time Markov Decision Process 8
3.2 Model-based Optimal Control 10

4 Benchmark Model **15**
4.1 The General ACE 15
4.2 ACE with DICE functional forms 18
4.3 Recursive Formulation 19

5 Mapping ACE onto DEQNs **22**
5.1 Policy Objective 22
5.2 Lagrange Conditions 23
5.3 Policy Network Input and Outputs 29

6 Results **32**
6.1 Network Architecture and Hyperparameters 32
6.2 Learning Statistics 35
6.3 Analytical Comparisons 37

7 Conclusion **42**

A DEQN Objective Details **49**
A.1 General ACE: Policy Objective Summary 49
A.2 Model-specific Mapping 49

B Simulation Initializations **51**
B.1 Exogenous variables 52
B.2 Endogenous Variables 56

C Supplementary Figures **57**

List of Tables

1 Common Activation Functions with Domain and Range (Chollet et al. 2015) 13
2 Network Architecture 32
3 Custom Functions 34
4 Training Scheme 35
5 Error Statistics ACE-DICE (2016 DICE), absolute errors $|\ell_m|$ 36
6 Error Statistics ACE-DICE (2023 DICE), absolute errors $|\ell_m|$ 37
7 Policy network (DEQN) against the analytical solution. Costates are averaged over the periods up to 2300; the abatement rate errors are taken over the same periods. 38
8 Labor Evolution Parameterization 52
9 TFP Evolution Parameterization 53
10 Carbon Intensity Evolution Parameterization 54
11 Abatement Cost Evolution Parameterization 55
12 Land Use Emissions Evolution Parameterization 55
13 Generic Parameterization of the Economy 56
14 Generic Parameterization of the Carbon Reservoirs 57
15 Generic Parameterization of Temperatures 57

List of Figures

1 Conventional modeling of agent-environment interaction. 9
2 Illustration of policy network architecture 33
3 Policy objective over training iterations (light line: per iteration; dark line: rolling median over 25 iterations) 36
4 Distribution of the absolute errors over the simulated episode. For conditions with several components, the largest component in each period is shown. Boxes show the interquartile range, whiskers the full range. 37
5 Consumption rate (top) and abatement rate (bottom): corrected DEQN, analytical solution and original implementation 39
6 SCC in USD/tCO$_2$ implied by the policy network and the analytical solution 40
7 Atmospheric temperature, atmospheric carbon and log capital along the trajectory simulated with the policy network and along the analytical optimal path 41
8 Exogenous law of motion for labor 52
9 Exogenous law of motion for tfp 53
10 Exogenous law of motion for carbon intensity 54
11 Exogenous law of motion for abatement costs 56
12 Costate variables $\lambda_{k,t}$, $\lambda_{M_1,t}$ and $\lambda_{\tau_1,t}$ estimated by the policy network along its simulated trajectory, against their constant analytical values. Note the narrow vertical ranges: the deviations are below 1.5% in all panels. 58

1 Introduction

Anthropogenic climate change, driven by greenhouse gas (GHG) emissions, constitutes an economic externality due to its significant economic damages. Integrated Assessment Models (IAMs) provide frameworks that model the interactions between economic policy and climate, helping to develop policies that mitigate these externalities. However, as IAMs increase in complexity, they can encounter computational challenges commonly referred to as the "curse of dimensionality" (Bellman 1961). With traditional solution methods, current IAMs often rely on restrictive assumptions to remain computationally tractable. This thesis aims to address these challenges by employing state-of-the-art deep learning methods that could potentially allow for more complex IAMs, reducing the need for restrictive assumptions.

One of the most notable applications of IAMs is in the assessment reports of the Intergovernmental Panel on Climate Change (IPCC), which evaluate the state of the global climate and inform policy decisions (Lee et al. 2023). IAMs have also been used to calculate the official social cost of carbon (SCC) for the United States (Interagency Working Group on the Social Cost of Carbon 2013), further underscoring their significance in economic policy formulation.

However, despite their extensive use, these models can face substantial computational challenges. As Azinovic et al. (2022) notes, these challenges arise when the system is stochastic, the state space is high-dimensional, there are nonlinearities in the equilibrium functions and complex distributions of state variables within the model's state space (ergodic set of states). Advancements in deep learning have begun to address these challenges, particularly within the realm of reinforcement learning (RL). These methods have been recently applied to and developed for economic models (Maliar et al. 2021; Azinovic et al. 2022; Valaitis et al. 2024).

This thesis extends the application of Deep Equilibrium Nets (DEQNs), as presented in Azinovic et al. (2022), by applying them to compute recursive economic equilibrium within the ACE model in a discrete-time setting. To the best of my knowledge, this work joins a narrow field of research using DEQNs for IAMs, where the only previously published papers are the ones by Folini et al. (2024) and Friedl et al. (2023).

Distinct from previous efforts that predominantly focused on DICE models, this thesis applies DEQNs to the Analytic Climate Economy (ACE) model, as presented in Traeger (2023). The advantage of applying this method to ACE is that it gives an analytic solution that can be used to analyze the quality of the numeric solution. Additionally, ACE is general in its structure, making my derivations easily extendable to other IAMs as well.

Furthermore, this thesis emphasizes the relationship between DEQNs and deep RL, a connection that has not been extensively explored in existing literature. This linkage is critical as it opens the door to integrating more advanced algorithms from the RL field, potentially improving existing methodologies for computing economic equilibria. Accompanying the thesis is a publicly available GitHub repository that serves as a comprehensive resource for future research in this area[1].

With correctly derived equilibrium conditions and a scaling of the objective functions that gives every condition comparable weight, the DEQN recovers the analytical solution of ACE with DICE functional forms closely: the consumption rate, the abatement rate, the costate variables and the implied social cost of carbon all match their closed-form values. The thesis also shows how sensitive the method is to the correctness of the equilibrium conditions. In the submitted version of this thesis, errors in the conditions on capital, consumption and emissions led the policy network towards a wrong fixed point, and the poor results were mistakenly attributed to the normalization of inputs.

The thesis is structured as follows: Section two provides a comprehensive review of the literature on IAMs related to ACE and explores computational methods for estimating economic equilibria, with a specific emphasis on RL. Section three delves into preliminary knowledge, covering RL concepts necessary to understand DEQNs. In section four, the general structure of the ACE model is discussed. Section five presents the necessary theoretical derivations required for mapping the general structure of ACE onto DEQNs. Section six discusses the results and findings. The thesis concludes in section seven. For detailed derivations of the specific models, supplementary figures and simulation initialization, please refer to the appendix of the thesis.

[1]The GitHub repository is available here: `https://github.com/hermanvest/DNN-for-ACE`

2 Literature

This thesis is related to two strands of the literature: I) the study of IAMs and II) computational methods for estimating economic equilibria. In this section, I will first examine the literature relevant to the particular IAM used in this study. Then, I will cover the relevant literature for DEQNs, specifically focusing on how it relates to RL.

2.1 Analytical Integrated Assessment Models

Nordhaus (1979) sparked an entire field of research into IAMs, including his own Dynamic Integrated Climate-Economy (DICE) model (Nordhaus 2011; Nordhaus 2018; Nordhaus et al. 2023). Simply put, there are two main branches of IAMs: numerical IAMs, often used in policy advising, and analytical IAMs, which offer analytical insights into important quantities such as the social cost of carbon (SCC). The IAM used in this thesis is the Analytical Climate Economy (ACE) model, presented in Traeger (2023). Therefore, the literature review will concentrate on those aspects of the literature that pertain directly to ACE[2].

Heal (1984) marks one of the first efforts to develop analytical frameworks for IAMs and focused on policy design under uncertainty, but was nonquantitative. Later came contributions such as Schauer (1995), who developed an explicit model of the SCC under uncertainty and Pizer (1999), who did a log-linearization of a DICE-based IAM. Other contributions were the linear quadratic models in Hoel et al. (2002) and Karp et al. (2012), but they did not model production- or energy sectors.

Golosov et al. (2014) made significant progress in the field by specifying a Dynamic Stochastic General Equilibrium (DSGE) model with an energy sector and modeled the production response to emissions. This was an extension of non-renewable resource models like the one found in Dasgupta et al. (1974). One limitation of Golosov et al. (2014) was that they assumed the production impulse response to emissions to be immediate and, therefore, also implicitly assumed an immediate response in temperatures. However, this is inconsistent with the observation that the oceans contribute to warming over centuries (Abraham et al. 2022). Gerlagh et al. (2018) extended the climate system representation to better model the persistence and delay from climate impacts.

Subsequent developments in Analytical Integrated Assessment Models (AIAMs), inspired by the work of Golosov et al. (2014), leveraged the property that the models that can

[2]For reviews on numerical IAMs, see Bosetti (2021) and Metcalf et al. (2017).

be transformed to a system linear in the states can be solved analytically, as pointed out in Karp (2017). A few notable applications of this approach include an application to international cooperation and intergenerational altruism in games that affect carbon levels (Karp 2017) and a non-cooperative multi-regional setting as in Hambel et al. (2021).

Traeger (2023) presented the Analytical Climate Economy (ACE) model, which addressed four key limitations in the literature. Firstly, ACE incorporates more realistic temperature dynamics by explicitly modeling the non-linear relationship between CO2 concentrations and temperature changes. Second, he improved upon the overly simplistic full-depreciation assumptions prevalent in Golosov et al. (2014). Third, by including delayed responses between emissions and their corresponding economic and climate impacts. Finally, ACE allowed for variable degrees of risk aversion distinct from intertemporal substitution elasticity, enhancing its utility in analyzing policy implications under uncertainty.

The following thesis applies a state-of-the-art deep learning technique called "Deep Equilibrium Nets", which is presented in the paper by Azinovic et al. (2022), to the analytical structure of ACE (Traeger 2023). The research work makes two important contributions to this field. Firstly, it represents one of the earliest applications to IAMs. Before this work, only two studies had applied this method to models based on DICE, as seen in Folini et al. (2024) and Friedl et al. (2023). This positions the present thesis at the front of deep learning applications to IAMs.

Secondly, by mapping DEQNs onto ACE, which is general in its structure, the thesis not only tests the quality and adaptability of the method but should also facilitate broader applications. Furthermore, DEQNs were introduced to overcome the curse of dimensionality (Bellman 1961), which is the exact problem IAMs face when incorporating more rich and complex features. Given that this application to ACE is generalizable, it paves the way for developing more complex IAMs.

2.2 Methods for Computing Economic Equilibrium

I will attempt to contextualize DEQNs (Azinovic et al. 2022) in the literature on economic equilibrium computation methods using Machine Learning (ML), emphasizing its relationship with the broader literature on solution algorithms that solve the kinds of problem formulations common in economics. I have found that this connection is often overlooked in literature reviews where DEQNs have been utilized, especially concerning its connection with Reinforcement Learning (RL). For a more comprehensive understanding of computational methods in economics, I suggest referring to the detailed survey on numerical methods in economics by Maliar et al. (2014) and the survey on solution

methods for DSGE models in Fernández-Villaverde (2016).

Smolyak sparse grids, a widely used method originally introduced by the Russian mathematician Smolyak (1963) have been frequently applied in an economic context, where several papers have demonstrated their efficacy in managing high-dimensional state spaces. For instance, Krueger et al. (2004) employed Smolyak sparse grids to tackle the computational challenges in solving overlapping generations (OLG) models with stochastic production. Winschel et al. (2010) applied Smolyak sparse grids in a Bayesian estimation framework for nonlinear dynamic models.

Brumm et al. (2017) introduced adaptive sparse grids. This advancement dynamically adjusts grid density in response to model complexity, allowing for even more precise and computationally efficient solutions to high-dimensional dynamic economic models. However, in their experiments, only going from solving a model with one asset to two assets was prohibitively difficult for them. The problem is that control variables need to be approximated on a hypercubic domain[3].

Traditional methods can struggle with dynamic models with stochasticity, high-dimensional state spaces, nonlinearities, and irregular state space geometries (Azinovic et al. 2022). Grid-free simulation-based methods can be used to overcome these issues. A paper by Judd et al. (2011) highlights the numerical stability and accuracy of stochastic simulation approaches for solving dynamic models, indicating that simulation-based approaches outperform grid-based methods in certain contexts. Maliar et al. (2015) merge simulation with projection methods to tackle high-dimensional problems, showcasing the flexibility and potential of combining these techniques to overcome the curse of dimensionality. Scheidegger et al. (2019) use a ML framework that leverages Gaussian processes and active subspaces to compute global solutions to dynamic stochastic economic models on irregular state spaces. This framework notably improves the ability to resolve local features and perform uncertainty quantification in high-dimensional settings, offering a significant advancement over traditional grid-based approaches.

Deep Neural Network (DNN) applications are becoming increasingly popular in algorithms used to estimate equilibrium in large economic models[4]. While tangential to the main focus of this thesis, it is also noteworthy that DNNs have demonstrated capability in overcoming the curse of dimensionality in approximating partial differential equations (PDEs). For instance, Jentzen et al. (2019) and Grohs et al. (2023), approximate Kolmogorov PDEs and Black-Sholes PDEs, respectively.

Additionally, a paper by Becker et al. (2020) demonstrates that DNNs can approximate

[3]Here, referring to the multi-dimensional grid representing the possible states of the system. Each axis represents a state variable, and the grid points represent the combinations of state variable values for which control needs to be computed.

[4]See Goodfellow et al. (2016) for an introduction to deep learning.

optimal stopping times to any given degree of accuracy, drawing inspiration from deep RL. Similarly, Gu et al. (2024) explores the integration of DNNs with Pontryagin's Maximum Principle (PMP), effectively recovering optimality conditions and optimal action sequences. The approach was successful in various RL environments.

RL is a branch of ML that studies intertemporal optimization and, therefore, closely aligns with the problem statement in economic modeling and naturally has many parallels to the algorithms used for solving such problems. RL can be categorized into two main types: model-free and model-based. In model-free RL, the agent learns to make decisions based solely on observed states and rewards without any underlying model of the environment's dynamics. Conversely, in model-based RL, the agent makes decisions informed by an understanding of the environment's dynamics. Most economic algorithms fall under the latter, including the approach in Azinovic et al. (2022) used in this thesis. For a thorough introduction to RL, I suggest referring to Sutton (2018), Shakya et al. (2023) for recent advancements, and to Achiam (2018) for codes and references to state-of-the-art RL papers and codes.

Close to the method in this thesis, Duarte et al. (2018), explicitly inspired by the RL literature, developed an algorithm for continuous-time finance that used DNNs combined with automatic differentiation and Ito's lemma to solve high-dimensional, nonlinear, continuous-time finance models. Another example of using DNNs in a continuous-time setting is presented in Fernández-Villaverde et al. (2023). They use DNNs to solve for the perceived law of motion (PLM) in a model with heterogeneous agents.

The use of objective functions based on Euler equations, first-order conditions, and Bellman residuals is common in the literature. Examples of such approaches can be found in studies by Valaitis et al. (2024), Maliar et al. (2021), and Azinovic et al. (2022), of which the last contains the solution algorithm used in the present thesis. All three studies present algorithms based on similar objectives and have similar algorithmic structures. Despite the similarities, Valaitis et al. (2024) claims to do supervised learning, Azinovic et al. (2022) claims it uses unsupervised learning, and Maliar et al. (2021) claims to be related to supervised, unsupervised and RL.

Despite the varied descriptions provided by the authors, the core methodologies in these studies largely align with the learning paradigm of RL, or more specifically, model-based RL. This alignment is evident as all three approaches rely on iterative, knowledge-based policy updates driven by first-order conditions, Euler equations, or Bellman residuals. These elements are fundamental to RL, where interactions with an environment generate the data and learning signals, allowing us to approximate an optimal policy parameterized by a neural network. For more information on how RL distinguishes itself from supervised- and unsupervised learning, please refer to Sutton (2018, p. 2).

To my knowledge, this kind of learning algorithm has few applications to IAMs. As mentioned, two papers to date have used DEQNs as presented in Azinovic et al. (2022). Folini et al. (2024) used DEQNs to systematically recalibrate the climate emulator of DICE, ensuring their alignment with state-of-the-art climate data from CMIP5 (Taylor et al. 2012). The paper by Friedl et al. (2023) employs DEQNs and Bayesian Learning to quantify uncertainty over the equilibrium climate sensitivity. Their method is applied to a high-dimensional IAM with stochastic climate tipping.

It is important to note that there have also been attempts to use model-free RL algorithms for IAMs. In one study, Strnad et al. (2019) applied different variations of deep Q-learning (Mnih et al. 2013) to the AYS model (as described in Heitzig et al. (2018)) and the model presented in Nitzbon et al. (2017). Recently, Wolf et al. (2023) explored more advanced RL algorithms, reward functions, and experimental setups to solve the AYS model.

This thesis makes several contributions to the literature. Firstly, it applies DEQNs to ACE, expanding the currently sparse literature that applies these methods to IAMs. Secondly, I provide a detailed explanation of the methods used, clarifying and strengthening the connection between DEQNs and RL. This can help future research in this area to draw on algorithms from the RL literature.

Third, to the best of my knowledge, this work is one of the first attempts to validate the quality of DEQN solutions against analytical solutions. The analysis of the numerical results reveals areas where solution quality is suboptimal; the thesis specifically identifies the need for improved normalization processes in both pre- and post-processing stages as critical for enhancing model performance.

Finally, the thesis is accompanied by a GitHub repository containing the codes and documentation for the results in the thesis. This repository is one of the few comprehensive, accessible resources available, which could significantly reduce the time and effort required for future research implementations.

3 Preliminaries

This section broadly reviews some basic concepts in RL and ML and aims to clarify and establish the connections between the RL literature and DEQNs. This discussion extends to approaches with similar methodologies, such as the ones in Valaitis et al. (2024) and Maliar et al. (2021). The section begins by introducing the agent-environment interface of RL and Bellman equations. Then, I will define the policy objective and how the method approximates it. Lastly, the section provides the complete pseudocode for the DEQN algorithm.

3.1 Discrete-time Markov Decision Process

In this subsection, I explore the Discrete-time Markov Decision Process (MDP) as a mathematical framework for modeling sequential decision-making problems within the context of RL. I focus on how the foundational elements of MDPs can be applied to economic decision-making problems. The subsequent discussion will detail the structure and components of MDPs, highlighting their relevance and adaptation to the economics domain.

RL uses an agent-environment interface as the basis for its mathematical notation. The "agent" refers to the entity responsible for selecting the control variables or actions, paralleling the role of economic agents in economic theory. Conversely, "environment" is used about the system that simulates forward the choice variables and returns the next state and reward; in the context of this thesis, this role is fulfilled by the IAM. Figure (1) shows how the dynamics of agent-environment interactions in RL are typically modeled. Please see Sutton (2018, Chapter 3) for a thorough introduction.

A Markov Decision Process (MDP) is a way to model sequential decision-making. Using the same notation as Sutton and Barto Sutton (2018, Chapter 3), the MDP is defined by the tuple $\mathcal{M} = \{\mathcal{S}, \mathcal{A}, \{P_{s,a}\}, \gamma, R\}$. Here, $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ represents a finite set of n states with the current state at any time t expressed as the vector $\boldsymbol{s}_t \in \mathbb{R}^n$. The set $\mathcal{A} = \{a_1, a_2, \ldots, a_k\}$ is a finite set of k actions with the action at time t represented by the vector $\boldsymbol{a}_t \in \mathbb{R}^k$. The function $P_{s,a}(\cdot)$ denotes the probability of transitioning to a subsequent state s', upon taking action a in state s. $\gamma \in (0, 1)$ denotes the discount factor. Lastly, the reward function $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ assigns a real-valued reward to each state-action pair.

This framework offers a generalized approach to modeling sequential decision-making in

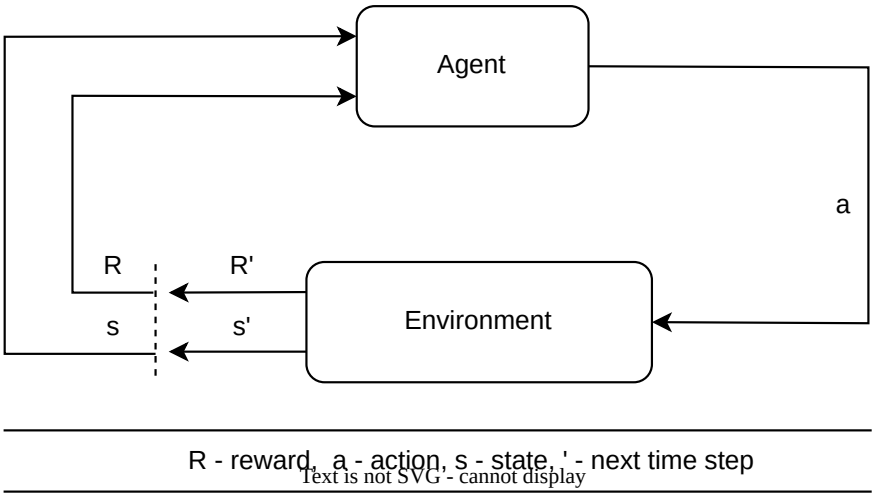


Figure 1: Conventional modeling of agent-environment interaction.

economics. To illustrate this connection more explicitly, consider this simple intertemporal optimization problem in economics. Suppose the state is represented by capital $s_t = K_t$ and that the economic agent at each time step t chooses the consumption level for the time step, meaning that the action is $a_t = C_t$. The agent is forward-looking and has time preferences to discount future utility $u(C_t) = r_t(s_t, a_t)$ with the discount factor $\gamma = \beta$. Additionally, the agent must consider uncertainty through transition probabilities in stochastic economic models.

In RL, the agent chooses actions according to a policy. For the purposes of this introduction, I will use a simplified notation, where the policy function is defined as a direct mapping from the set of states $\mathcal{S}$ to the set of actions $\mathcal{A}$ represented as $\pi : \mathcal{S} \to \mathcal{A}$[5]. The value function for a policy π can be evaluated at any state s_t and is given by:

$$V_\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right], \quad \forall s \in \mathcal{S}, \tag{3.1}$$

where the expectation $\mathbb{E}_\pi$ is over state transitions and rewards, and the reward function is expressed as R_t for brevity. This expression evaluates the sum of expected discounted rewards obtained by following the policy π from state s. This recursive relationship is foundational to the Bellman equation for the value function. Introduced in the seminal work in Bellman (1957), the Bellman equation can be expressed as follows:

$$V_\pi(s_t) = \mathbb{E}_\pi[R_{t+1} + \gamma V(s_{t+1})] \tag{3.2}$$

[5]This represents a simplification from the typical stochastic policy functions in reinforcement learning (which you will find throughout Sutton (2018) as well), where a policy is normally defined as a mapping from states and actions to a probability distribution: $\pi : \mathcal{S} \times \mathcal{A} \to [0, 1]$. That means $\pi(a_t|s_t) = Pr(A_t = a_t|S_t = s_t)$.

The objective of RL is to identify a policy that maximizes the rewards across all states, maximizing the value function. This is encapsulated in the principle of Bellman optimality, which can be mathematically represented as:

$$\pi^*(s) = \arg\max_a \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s, A_t = a], \tag{3.3}$$

where π^* denotes the optimal policy. To simplify and make my notation more compact, I will also introduce the action-value function $Q_\pi(\boldsymbol{s}_t, \boldsymbol{a}_t)$, which is the right-hand side of the Bellman equation:

$$Q_\pi(s, a) = \mathbb{E}[R_{t+1} + \gamma V_\pi(S_{t+1}) \mid S_t = s, A_t = a] \tag{3.4}$$

This function quantifies the expected return starting from state s, taking action a, and thereafter following policy π.

3.2 Model-based Optimal Control

This section aims to formalize the application of DEQNs (Azinovic et al. 2022) in Folini et al. (2024) and Friedl et al. (2023). To achieve this, we need to cover two things: the policy objective and function approximation with DNNs. I will use the notation we have developed in the previous section to derive the policy objective. Additionally, if you are familiar with deep learning, feel free to skip the section on function approximation with DNNs.

3.2.1 The Policy Objective

The policy objective in Folini et al. (2024) and Friedl et al. (2023) relies on known system dynamics, where the next state variables are given by functions f of the current state and action vectors:

$$\boldsymbol{s}_{t+1} = f(\boldsymbol{s}_t, \boldsymbol{a}_t), \quad \boldsymbol{s}_0 \text{ given} \tag{3.5}$$

We want to use the knowledge of the underlying dynamics of the environment to find the optimal policy π^* that maximizes the action-value function under the constraints of the dynamics of the environment, given an initial state $\boldsymbol{s}_0$:

$$\begin{aligned} \pi^*(s) &= \arg\max_{\boldsymbol{a}_t} Q_\pi(\boldsymbol{s}_t, \boldsymbol{a}_t) \\ s.t. \quad &\text{Eq. (3.5)}, \end{aligned} \tag{3.6}$$

This problem statement is similar to the papers by Friedl et al. (2023) and Folini et al. (2024). However, Folini et al. (2024) did not employ the value function in their approach as they had a deterministic model. Although I also have a deterministic model, I have chosen to use the value function as it yields the same result, and the general approach can be applied to a stochastic setting.

Both papers use a Lagrangian formulation of the optimization problem, where the objective is to maximize the action-value function subject to the model's equations of motion. This formulation offers a set of necessary conditions that the optimal trajectory $\tau^* = \{\boldsymbol{s}^*_{0:T}, \boldsymbol{a}^*_{0:T}\}$ must satisfy the optimal control problem (eq. 3.6). To introduce these necessary conditions, I will write the Lagrangian as:

$$\mathcal{L}(\boldsymbol{s}_t, \boldsymbol{a}_t, \boldsymbol{\lambda}_{t+1}) = Q_\pi(\boldsymbol{s}_t, \boldsymbol{a}_t) + \boldsymbol{\lambda}^\top_{t+1} \left(f(\boldsymbol{s}_t, \boldsymbol{a}_t) - \boldsymbol{s}_{t+1} \right). \tag{3.7}$$

In the above equation, $\top$ denotes transposition, and $\boldsymbol{\lambda}_t$ represents the costate variables, which are also known as the shadow price in economics. However, in this thesis, I will mainly refer to them as costate variables since they are used for computation purposes. The optimal control must satisfy the Lagrange conditions:

$$\boldsymbol{\lambda}^*_t = \frac{\partial \mathcal{L}(\boldsymbol{s}^*_t, \boldsymbol{a}^*_t, \boldsymbol{\lambda}^*_{t+1})}{\partial \boldsymbol{s}_t} \tag{3.8}$$

$$\frac{\partial \mathcal{L}(\boldsymbol{s}^*_t, \boldsymbol{a}^*_t, \boldsymbol{\lambda}^*_{t+1})}{\partial \boldsymbol{a}_t} = 0 \tag{3.9}$$

The Lagrange conditions define the model's first-order necessary conditions for optimality. We assume full information of the control Lagrangian along with its first-order information with respect to state and control variables. This allows us to simulate the system forward and use the Lagrange conditions as the policy objective for a function approximator.

It is worth noting that in the paper by Azinovic et al. (2022), what I refer to as the policy objective here is referred to as the loss function in their paper. These terms can be used interchangeably, but since this method belongs to the learning paradigm of RL, it is more accurately called a policy objective.

The policy objective is fitted using regression through mean squared error (MSE). Each simulated trajectory will be divided into B batches of size b. The policy objective is then

evaluated at each point along the sampled batches from the trajectory:

$$\ell_\theta \doteq \frac{1}{b} \sum_{i=1}^{b} \sum_{m=0}^{M} \left[\ell_m \left(\boldsymbol{s}_i, \pi_\theta(\boldsymbol{s}_i) \right) \right]^2, \tag{3.10}$$

where m indexes the Lagrange condition and i indexes the state sample from the trajectory and π_θ is a parameterized policy function.

3.2.2 Function Approximation

Given that we are dealing with continuous state and action spaces, exact storage of the policy is not feasible, as we don't have an infinite amount of memory. As highlighted in Sutton (2018, p. 197), function approximation introduces a significant advantage in this context. It allows the policy function to be represented as a parameterized functional form by a parameter vector $\boldsymbol{\theta} \in \mathbb{R}^d$ instead of a table or grid. This ensures that the dimensionality of the parameter vector is considerably smaller than that of the state space $\mathbb{R} << d$, thus allowing adjustments to the policy across numerous states simultaneously.

The DEQN algorithm uses a vanilla feed-forward neural network (FNN), also known as a multi-layer perception (MLP). The parameter vector $\boldsymbol{\theta}$ consists of weight vectors $\boldsymbol{W}_i$ and bias vectors $\boldsymbol{b}_i$ for each i-th layer of the MLP, where $i \in \{1, ..., N\}$, $N \in \mathbb{N}$. This structure enables the MLP to approximate a wide range of functions by adjusting the parameters. The mathematical representation of the parameterized policy function is defined as follows:

$$\pi_{\boldsymbol{\theta}}(\boldsymbol{s_t}) = \mathcal{N}(\boldsymbol{s_t}) = \boldsymbol{\sigma}_N(\boldsymbol{W}_N \boldsymbol{\sigma}_{N-1}(\boldsymbol{W}_{N-1} ... \boldsymbol{\sigma}_1(\boldsymbol{W}_1 \boldsymbol{s_t} + \boldsymbol{b}_1) \dots) + \boldsymbol{b}_{N-1}) + \boldsymbol{b}_N), \tag{3.11}$$

where $\boldsymbol{\sigma}_i$ are activation functions used at each layer i. From here, I will refer to the MLP parameterizing the policy function as the policy network. Generally, activation functions introduce non-linearity into neural networks, enabling them to model complex relationships in the data. They essentially filter and transform the incoming data, enabling networks to learn features at different levels of abstraction. Table (1) lists some commonly used activation functions.

The policy network's parameters are updated using Stochastic Gradient Descent (SGD)[6]. This involves evaluating the outputs of the policy network on the policy objective (eq. 3.10) and taking the gradients of it. Let $\nabla \ell_\theta$ denote the column vector of partial derivatives with respect to the parameters of the policy network:

[6]For a thorough intro to SGD in an RL context, see Sutton (2018, Chapter 9-11)

Name	**Form**	**Domain**	**Range**
ReLU	$f(x) = \max(0, x)$	$x \in \mathbb{R}$	$f(x) \in [0, \infty)$
Leaky ReLU	$f(x) = \max(\alpha x, x)$	$x \in \mathbb{R}$, α small	$f(x) \in (-\infty, \infty)$
Sigmoid	$f(x) = \frac{1}{1+e^{-x}}$	$x \in \mathbb{R}$	$f(x) \in (0, 1)$
Tanh	$f(x) = \frac{\exp(x)-\exp(-x)}{\exp(x)+\exp(-x)}$	$x \in \mathbb{R}$	$f(x) \in (-1, 1)$
Softplus	$f(x) = \ln(1 + e^x)$	$x \in \mathbb{R}$	$f(x) \in (0, \infty)$
ELU	$f(x) = \begin{cases} x & \text{if } x > 0 \\ \alpha(e^x - 1) & \text{otherwise} \end{cases}$	$x \in \mathbb{R}, \alpha > 0$	$f(x) \in (-\alpha, \infty)$

Table 1: Common Activation Functions with Domain and Range (Chollet et al. 2015)

$$\nabla \ell_\theta \doteq \begin{bmatrix} \frac{\partial \ell_\theta}{\partial \theta_1} & \frac{\partial \ell_\theta}{\partial \theta_2} & \dots & \frac{\partial \ell_\theta}{\partial \theta_d} \end{bmatrix}^\top \tag{3.12}$$

When we update the network parameters, we aim to take the step that moves us most toward the optimal path, effectively minimizing the policy objective (eq. 3.10). We, therefore, update the parameter vector with the negative of the gradient, hence "gradient descent" in SGD. Additionally, the updating rule includes a small coefficient α that ensures updates are not too large. The updating step is defined as follows:

$$\boldsymbol{\theta} \doteq \boldsymbol{\theta} - \alpha \nabla \ell_{\boldsymbol{\theta}} \tag{3.13}$$

In practice, an optimizer performs the updating step. The ADAM optimizer in Kingma et al. (2017) is widely used in deep learning and was also used in Folini et al. (2024) and Friedl et al. (2023).

3.2.3 Algorithm

Putting it all together, we arrive at algorithm (1). The algorithm's requirements are listed under "Require" and the output is given under "Ensure". Following that, the pseudocode outlines the sequence of operations, starting with parameter initializations. The algorithm's main loop generates episodes and processes them through epochs, where each batch is evaluated to update the policy network using SGD.

Algorithm 1 DEQN policy iteration for estimating $\pi \approx \pi^*$

Require:

A differentiable policy objective $\ell : \mathcal{S} \times \mathcal{A} \times \mathbb{R}^d \to \mathbb{R}$

Algorithm parameters: Episode length T, MaxIterations, NumEpochs, step size parameter α, and error tolerance ϵ

Starting state $\boldsymbol{s}_0 \in \mathbb{R}^{|\mathcal{S}|}$

Ensure:

Trained parameters $\boldsymbol{\theta}$ of the policy network s.t. $\boldsymbol{\pi}_{\boldsymbol{\theta}}(\boldsymbol{s}_t)$ minimizes the policy objective.

Initialize parameters $\boldsymbol{\theta} \in \mathbb{R}^d$ arbitrarily for the policy network $\boldsymbol{\pi}_{\boldsymbol{\theta}}(\boldsymbol{s}_t)$
Initialize episode counter $\leftarrow 0$
Initialize $\Delta \leftarrow \infty$

while counter $<$ MaxIterations and $\epsilon < \Delta$ **do**
    Generate an episode of length T with trajectory τ following $\boldsymbol{\pi}_{\theta}$
    $\Delta \leftarrow 0$
    for epoch $= 1$ to NumEpochs **do**
        List of batches $\leftarrow$ divide episode into batches
        for batch in list of batches **do**
            Compute policy objective $\ell_{\boldsymbol{\theta}}$ for batch
            $\Delta \leftarrow \max(\Delta, |\ell_{\boldsymbol{\theta}}|)$
            Do batch gradient descent $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla \ell_{\boldsymbol{\theta}}$
        end for
    end for
    counter $\leftarrow$ counter $+1$
end while
return $\boldsymbol{\theta}$

4 Benchmark Model

First, this section revisits the general ACE with the DICE-inspired climate emulator. Then, it will cover the case of ACE-DICE, using the functional forms of the DICE model, based on the discussion in Traeger (2021). As I merely cover the key points in his model, I refer you to Traeger (2023) and Traeger (2021) for a more thorough discussion of the model and possible extensions.

4.1 The General ACE

4.1.1 The Economic Model

The final net output is given by final gross output Y_t, minus the fraction of output destroyed at each time step from climate-related damages, given by the function D of atmospheric temperature above preindustrial level $T_{1,t}$. Let final gross output be a function F of vectors of technology $\boldsymbol{A_t}$, labor $\boldsymbol{N_t}$, capital $\boldsymbol{K_t}$ and resource inputs $\boldsymbol{E_t}$, where the input vectors are of dimensions $I_j \in \mathbb{N}$ with $j \in \{A, N, K, E\}$ such that $F : \mathbb{R}^{I_A} \times \mathbb{R}^{I_N} \times \mathbb{R}^{I_K} \times \mathbb{R}^{I_E} \to \mathbb{R}$. Then,

$$Y_t^{\text{net}} = F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t)[1 - D(T_{1,t})], \quad \text{where} \tag{4.1}$$

$$F(\boldsymbol{A}_t, \boldsymbol{N}_t, \gamma^\kappa \boldsymbol{K}_t, \boldsymbol{E}_t) = \gamma^\kappa F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t), \quad \forall \gamma \in \mathbb{R}_+$$

is net output where gross total output is a function homogenous of degree κ in capital. Aggregate labor and capital are assumed to be optimally distributed across sectors such that $\sum_{i=1}^{I_N} N_{i,t} = 1$ and $\sum_{i=1}^{I_K} K_{i,t} = K_t$. The resource vector's first $I^d \leq I_E$ elements are (dirty) fossil fuels that emit CO_2 such as oil, coal, and gas. The rest of the resource input vector is renewable energy inputs such as water, wind, and sunlight and are indexed from I^{d+1} to I_E. These are assumed to be abundant[7]. Finally, emissions from land conversion, forestry, and agriculture are denoted as $\boldsymbol{E}_{\boldsymbol{t}}^{\text{exo}}$ and are assumed to be exogenous.

This thesis will use the damage function that permits an analytical solution. It is not required for this solution method to work, but it will allow for qualitatively evaluating the quality of the solution method used in this thesis. The function is defined as

[7]The original model includes a resource stock. I have chosen not to include the resource stock as a state variable for simplicity, but the extension would be straightforward.

$$D(T_{1,t}) = 1 - \exp(-\xi_0 \exp[\xi_1 T_{1,t}] + \xi_0), \quad \xi_0 \in \mathbb{R}, \tag{4.2}$$

where $\xi_1 \approx \frac{1}{4}$ and ξ_0 is a free damage parameter. When atmospheric temperatures are at the preindustrial levels, $T_{1,\text{pre}} = 0$, damage to output is zero: $D(T_{1,\text{pre}}) = 0$.

Investments are modeled as the residual output after accounting for climate damages and consumption:

$$I_t = Y_t \left[1 - D(T_{1,t})\right] - C_t \tag{4.3}$$

Finally, for the capital equation of motion, I will be using the approximate model of standard capital persistence, where aggregate capital accumulates as:

$$K_{t+1} = I_t \left[\frac{1 + g_{k,t}}{\delta_k + g_{k,t}}\right] \tag{4.4}$$

Here, $g_{k,t}$ is the capital growth and δ_k is the capital depreciation.

4.1.2 The Climate Model

I will use the ACE-DICE style carbon cycle model, as it is widely used in climate emulators of IAMs[8]. The model consists of $m \in \mathbb{N}$ carbon reservoirs, represented by the vector $\boldsymbol{M}_t \in \mathbb{R}^m$:

$$\boldsymbol{M_t} = \begin{bmatrix} M_{1,t} & M_{2,t} & \dots & M_{m,t} \end{bmatrix}^\top. \tag{4.5}$$

The reservoirs are ordered from the highest to deepest reservoirs, with the upper layers starting from the atmosphere, $M_{1,t}$, having the lower indexes, and the deeper layers are indexed higher. The matrix $\boldsymbol{\Phi}$ is the transition matrix, and contains elements $\Phi_{i,j}$ which are the transfer coefficient from reservoir j to i. The equation of motion for the carbon reservoirs is as follows:

$$\boldsymbol{M}_{t+1} = \boldsymbol{\Phi} \boldsymbol{M}_t + \boldsymbol{b} \left(\sum_{i=1}^{I_d} E_{i,t} + E_t^{\text{exo}} \right). \tag{4.6}$$

Originally, $\boldsymbol{b} \in \mathbb{R}^m$ contains coefficients that distribute emissions into the different carbon reservoirs. For simplicity, I will assume that $\boldsymbol{b}$ is the first unity vector $\boldsymbol{b} = \boldsymbol{e_1}$ for the

[8]Traeger (2023) presents two models: ACE-DICE and ACE-Joos. ACE-Joos is based on Joos et al. (2013) and addresses recent criticism of DICE's carbon cycle using an impulse-response model of atmospheric carbon that is calibrated to an ensemble of climate models (Taylor et al. 2012)

rest of the thesis. Emissions consist of the first I_d indices in $\boldsymbol{E_t}$ and exogenously given emissions E_t^{EXO}.

When emissions are released into the atmosphere, the concentration of atmospheric carbon and other greenhouse gases (GHGs) increases, disrupting the energy balance of our planet and leading to a rise in global temperatures. This heating process, known as anthropogenic radiative forcing, happens immediately when GHGs enter the atmosphere. In ACE, radiative forcing is a function of atmospheric concentration of CO_2 $M_{1,t}$, exogenous forcing from other non-CO_2 GHGs G_t and preindustrial carbon concentration M_{pre}:

$$F_t = \eta \frac{\log \frac{M_{1,t}+G_t}{M_{\text{pre}}}}{\log 2}. \tag{4.7}$$

The parameter η quantifies the intensity of the greenhouse effect, where forcing immediately increases by η if CO_2 concentrations double. Conversely, the temperature has a very slow response to heating. It takes decades to centuries to reach a new temperature equilibrium in the lower atmosphere $T_{1,\text{eq}}^{\text{new}}$ in response to a new forcing level F^{new}. The new equilibrium is characterized as follows:

$$T_{1,\text{eq}}^{\text{new}} = \frac{s}{\eta} F^{\text{new}} = \frac{s}{\log 2} \log \frac{M_{1,\text{eq}} + G_{\text{eq}}}{M_{\text{pre}}}. \tag{4.8}$$

Here, the climate sensitivity parameter s represents the amount by which global temperatures are expected to increase if the CO_2 concentrations in the atmosphere are doubled from preindustrial levels.

The temperature layers are organized in adjacent layers $T_{i,t}$ $i \in \{0, 1, 2, ..., l,\ l \in \mathbb{N}\}$ where index zero is reserved for the atmospheric equilibrium temperature:

$$T_{0,t} = \frac{s}{\eta} F_t. \tag{4.9}$$

Each temperature layer exchanges heat with the adjacent layers. For an analytic solution, the next period temperature in layer i is modeled as a mean of layer $i \in \{1, ..., l\}$ current temperature and its adjacent layers $T_{i-1,t}$ and $T_{i+1,t}$:

$$\begin{aligned} T_{i,t+1} = &\frac{1}{\xi_1} \log \Big((1 - \sigma_{i,i-1} - \sigma_{i,i+1}) \exp[\xi_1 T_{i,t}] \\ &+ \sigma_{i,i-1} \exp[\xi_1 T_{i-1,t}] + \sigma_{i,i+1} \exp[\xi_1 T_{i+1,t}] \Big) \end{aligned} \tag{4.10}$$

with weight matrix $\boldsymbol{\sigma}$ capturing the heat transitions between the layers.

4.2 ACE with DICE functional forms

This section explores a potential specification of the general ACE, utilizing the functional form of the production function in DICE, its carbon cycle, and temperature layers. It is based on Traeger (2021). The section begins by demonstrating how we can specify ACE using the DICE production function and then delves into the relevant details of the climate model.

4.2.1 The Production Function

This section reviews how the production function of the general ACE can be specified to cover the DICE production function. Traeger (2021) specifies an energy-enriched production system with Cobb-Douglas production. The production function is amended by introducing energy sectors represented by a function G of vectors of exogenous technologies $\boldsymbol{A}_t^E$, allocated labor $\boldsymbol{N}_t^E$ and energy inputs $\boldsymbol{E}_t$. Then,

$$Y_t = F(\boldsymbol{A_t}, \boldsymbol{N_t}, \boldsymbol{K_t}, \boldsymbol{E_t}) = \underbrace{A_t K_t^{\kappa} N_t^{\eta}}_{\text{DICE's } Y_t^{\text{gross}}} G(\boldsymbol{A_t^E}, \boldsymbol{N_t^E}, \boldsymbol{E_t}), \tag{4.11}$$

captures final output with the traditional Cobb-Douglas levels of technology A_t, optimally allocated labor N_t and capital K_t. The parameter η represents the labor elasticity of substitution and is $\eta = 1 - \kappa$ in the case of DICE. Any version of DICE uses the Cobb-Douglas production function for gross output. Thus, the functional form for gross output in DICE, Y_t^{gross}, is already covered by equation (4.11).

DICE models the energy system implicitly using the abatement cost function $\Lambda_t(\cdot)$. In any version of DICE, economic production produces CO_2 emissions, constituting an externality. In DICE, the social planner can mitigate the externality by choosing an abatement rate $\mu_t \in [0, 1]$, which reduces total industrial emissions. That way, business as usual (BAU) industrial emissions are reduced by the fraction $1 - \mu_t$, such that total industrial emissions at abatement are:

$$E_t = (1 - \mu_t)\sigma_t Y_t^{\text{gross}} \tag{4.12}$$

where BAU emissions are assumed to be proportional to gross output, scaled by σ_t, the carbon intensity of production that is exogenously given. BAU emissions are therefore $E_t^{BAU} = \sigma_t Y_t^{\text{gross}}$. You will notice that equation (4.11) does not use an abatement rate as input. To unite the two production models, Traeger (2021) rewrites the emissions equation (4.12) to allow us to arrive at an explicit model of the energy system:

$$\mu_t = 1 - \frac{E_t}{E_t^{\text{BAU}}} \tag{4.13}$$

Finally, abatement costs depend on the abatement rate through the following function (inserting equation (4.13) for μ_t):

$$\Lambda(\mu_t) = \theta_{1,t}\mu_t^{\theta_2} = \theta_{1,t}\left(1 - \frac{E_t}{E_t^{\text{BAU}}}\right)^{\theta_2}, \tag{4.14}$$

where $\theta_{1,t}$ is an exogenous reduction in abatement costs over time and the parameter $\theta_2 = 2.6$ in both DICE 2016 and DICE 2023. The abatement costs are proportional to output, meaning that it costs the economy $Y_t^{\text{gross}}\Lambda_t(\cdot)$ to abate. Without climate damages accounted for, the net output is $Y_t^{\text{gross}}(1-\Lambda_t(\cdot))$. If we let $A_t^E = \theta_{1,t}$ the production system is given as[9]:

$$G(A_t^E, E_t^{\text{BAU}}, E_t) = 1 - \theta_{1,t}\left(1 - \frac{E_t}{E_t^{\text{BAU}}}\right)^{\theta_2}. \tag{4.15}$$

Putting it all together, the final production function is:

$$Y_t = A_t K_t^{\kappa} N_t^{1-\kappa}\left[1 - \theta_{1,t}\left(1 - \frac{E_t}{E_t^{\text{BAU}}}\right)^{\theta_2}\right]. \tag{4.16}$$

4.2.2 The Climate Model

For the purposes of this thesis, I chose to lie close to the climate emulator in DICE. DICE has a carbon cycle with three reservoirs denoted as $\mathbf{M}_t \in \mathbb{R}^3$. This models atmospheric content $M_{1,t}$, upper ocean and biosphere $M_{2,t}$, and finally the deep ocean $M_{3,t}$. As a result, the transition matrix is a 3x3 matrix $\mathbf{\Phi} \in \mathbb{R}^{3\times 3}$. Finally, I will be using a two-layer temperature model, which means that the temperature vector is two dimensional: $\boldsymbol{T}_t \in \mathbb{R}^2$.

4.3 Recursive Formulation

The objective of the social planner is to solve the discrete time, infinite horizon problem with a discount factor $\beta < 1$ and welfare defined as the logarithm of aggregate consump-

[9]Deviating from Traeger (2021), as this notation is simpler and his notation is used for further derivations where his functional form is required, where it is not in my case.

tion C_t:

$$\max \sum_{t=0}^{\infty} \beta^t \log C_t \tag{4.17}$$

I will follow Traeger (2023) in using the transformed model that is linear in the states' equations of motion. For this, I refer you to the online appendix, section A3, accompanying his paper.

The transformed temperatures are defined as $\tau_{i,t} = \exp(\xi_1 T_{i,t}),\ i \in \{1, \ldots, l\}$. The equation of motion of the transformed temperatures is:

$$\boldsymbol{\tau}_{t+1} = \boldsymbol{\sigma}\boldsymbol{\tau}_t + \sigma^{\text{forc}} \frac{M_{1,t} + G_t}{M_{\text{pre}}} \boldsymbol{e}_1. \tag{4.18}$$

Using the damage function and transformed temperatures, he defines welfare as a function of the consumption rate:

$$\begin{aligned} u(x_t) \equiv \log(C_t) &= \log(x_t) + \log(Y_t) + \log[1 - D(T_{1,t})] \\ &= \log x_t + \log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t) - \xi_0 \tau_{1,t} + \xi_0 \end{aligned} \tag{4.19}$$

Furthermore, he uses $\log K_t = k_t$, where the transforming the equation of motion for the effective capital stock to:

$$\begin{aligned} k_{t+1} = &\log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t) - \xi_0 \tau_{1,t} + \xi_0 + \log(1 - x_t) \\ &+ \log[1 + g_{k,t}] - \log[\delta_k + g_{k,t}] \end{aligned} \tag{4.20}$$

Let the state vector $\boldsymbol{s}_t$ capture the state variables in the general ACE-DICE model as such $\boldsymbol{s}_t = \begin{bmatrix} k_t & \boldsymbol{M}_t^\top & \boldsymbol{\tau}_t^\top \end{bmatrix}$. Then we get the following Bellman equation in terms of transformed state variables:

$$V(\boldsymbol{s}_t) = \max_{x_t, \boldsymbol{E_t}} \left\{ \log x_t + \log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t) - \xi_0 \tau_{1,t} + \xi_0 + \beta V(\boldsymbol{s}_{t+1}) \right\} \tag{4.21}$$

Two important results follow from this recursive formulation. The first is that it provides an analytical solution for the consumption rate. This relationship is derived in Traeger (2023), establishing a constant relationship across all time steps:

$$x_t^* = x^* = 1 - \beta\kappa. \tag{4.22}$$

The second result is that it allows for a closed-form solution of the optimal abatement rate in the DICE variation of ACE, as presented in section (4.2). Traeger (2021) derives the optimal abatement rate as:

$$\mu_t = \left(\frac{\Gamma_t}{p_t^{\text{back}}[1 - D(T_{1,t})]} \right)^{\frac{1}{\theta_2 - 1}}. \tag{4.23}$$

In this formulation, p_t^{back} is the backstop price in USD per tCO_2 and Γ_t represents the total social cost associated with using a CO_2-measured unit of fossil fuel in period t. For my analysis, Γ_t equals the SCC in period t, denoted in USD/tCO_2. Notably, while Traeger (2021) defines Γ_t as the sum of the Hotelling rent and the SCC ($\Gamma_t = HOT_t + SCC_t$), my model does not rely on any hotelling rent. The equation for the the SCC in period t, is given as:

$$SCC_t = 1000\,\zeta\,\frac{\beta^2 Y_t^{net}}{M_{pre}}\xi_0\,\underbrace{[(\mathbf{1} - \beta\boldsymbol{\sigma})^{-1}]_{1,1}}_{\approx 2.1460}\,\sigma^{forc}\,\underbrace{[(\mathbf{1} - \beta\boldsymbol{\Phi})^{-1}]_{1,1}}_{\approx 5.1896}, \tag{4.24}$$

where $\beta = (1+\rho)^{-\Delta}$ is the discount factor per period of $\Delta = 5$ years. The factor $1000\,\zeta$ converts the expression into USD per tCO_2: net output Y_t^{net} is measured in trillion USD per year, and $\zeta = \Delta/3.664$ is the increase in atmospheric carbon (GtC) from one $GtCO_2$ of yearly emissions over a period (see Appendix A). Expressed through the costate variables of section (5), the same SCC is $SCC_t = -1000\,Y_t^{net}\,\zeta\lambda_{M_1,t}/(1+\lambda_{k,t})$, which allows the SCC implied by the policy network to be compared with the analytical value.

where $\mathbf{1}$ is the identity matrix, -1 indicates inversion of the matrix, and the subscript [1,1] specifies the element located at the first row and first column of the inverted matrix.

5 Mapping ACE onto DEQNs

This section maps the general ACE-DICE onto the DEQN solution framework. The derivations largely follow those taken in the appendices in Folini et al. (2024) and Friedl et al. (2023), but is more comprehensive to facilitate understanding and easier replication for adaptations to new models. For details on model-specific mappings needed for the experiments conducted in this study, please refer to the thesis' appendix (A).

This section begins by establishing the policy objective through the recursive Lagrange problem. Next, I derive the first-order necessary Lagrange conditions and move on to the envelope conditions, which help eliminate the value function derivatives in the first-order conditions. The section then formulates the final objective functions that guide the decision-making process. Finally, it provides details of the state- and action space as well as normalizations.

5.1 Policy Objective

Recall from equation (3.6), that the action value function $Q(\boldsymbol{s}_t, \boldsymbol{a}_t)$ represents the right-hand side of the Bellman equation. Define the action vector $\boldsymbol{a}_t \in \mathbb{R}^{1+I_E}$ as $\begin{bmatrix} x_t & \boldsymbol{E}_t^\top \end{bmatrix}$. Then, the action-value function for the general ACE model, as given in equation (4.21), is expressed as:

$$Q(\boldsymbol{s}_t, \boldsymbol{a}_t) = \log x_t + \log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t) - \xi_0 \tau_{1,t} + \xi_0 + \beta V(\boldsymbol{s}_{t+1}).$$

We want to determine the optimal policy π^* that maximizes this action-value function, constrained by the dynamics governing the system's states:

$$\pi^*(\boldsymbol{s}_t) = \underset{x_t, \boldsymbol{E_t}}{\arg\max} \left\{ \log x_t + \log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t) - \xi_0 \tau_{1,t} + \xi_0 + \beta V(\boldsymbol{s}_{t+1}) \right\} \tag{5.1}$$

$$
\begin{aligned}
\text{s.t.} \quad & K_{t+1} = Y_t \left[1 - D(T_{1,t}) - c_t\right] \left[\frac{1 + g_{k,t}}{\delta_k + g_{k,t}}\right] \\
& \boldsymbol{M}_{t+1} = \boldsymbol{\Phi} \boldsymbol{M}_t + \boldsymbol{e}_1 \left(\sum_{i=1}^{I^d} E_{i,t} + E_t^{\text{exo}} \right) \\
& \boldsymbol{\tau}_{t+1} = \boldsymbol{\sigma} \boldsymbol{\tau}_t + \boldsymbol{e}_1 \sigma^{\text{forc}} \frac{M_{1,t} + G_t}{M_{\text{pre}}} \\
& \boldsymbol{E_t} \geq \boldsymbol{0}
\end{aligned}
$$

To derive the necessary conditions, we need to associate costate variables with each equation of motion. Define $\lambda_{k,t}$ as the costate variable for the capital equation of motion. Similarly, let $\boldsymbol{\lambda}_{M,t}$ be a vector of costate variables corresponding to the dynamics of the carbon reservoirs, where each carbon reservoir $M_{i,t+1}$ has the corresponding $\lambda_{M_i,t}$. Costate variables $\lambda_{\tau_i,t}$ correspond to each temperature layers $\tau_{i,t+1}$. For the constraints on non-negative emissions, $\boldsymbol{\lambda}_{E,t}$ encompasses the costate variables $\lambda_{E_i,t}$ tied to $E_{i,t}$. These costate variables are collected into a single vector $\boldsymbol{\lambda_t}$ and are expressed as $\boldsymbol{\lambda}_t = \begin{bmatrix} \lambda_{k,t} & \boldsymbol{\lambda}_{M,t}^\top & \boldsymbol{\lambda}_{\tau,t}^\top & \boldsymbol{\lambda}_{E,t}^\top \end{bmatrix}^\top$. The next-period states $\boldsymbol{s}_{t+1}$ are treated as choice variables tied to today's states and controls through the equations of motion, so that the effect of today's controls on the continuation value runs through the costate variables. The Lagrangian formulation of the recursive problem stands as:

$$
\begin{aligned}
\mathcal{L}(\boldsymbol{s}_t, \boldsymbol{a}_t, \boldsymbol{\lambda}_t) = \max_{x_t, \boldsymbol{E}_t, \boldsymbol{s}_{t+1}} \; & \log x_t + \log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t) - \xi_0 \tau_{1,t} + \xi_0 + \beta V(\boldsymbol{s}_{t+1}) \\
& + \lambda_{k,t} \Big\{ \log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t) - \xi_0 \tau_{1,t} + \xi_0 + \log(1 - x_t) \\
& + \log[1 + g_{k,t}] - \log[\delta_k + g_{k,t}] - k_{t+1} \Big\} \\
& + \boldsymbol{\lambda}_{M,t}^\top \left\{ \boldsymbol{\Phi} \boldsymbol{M}_t + \boldsymbol{e}_1 \left(\sum_{i=1}^{I^d} E_{i,t} + E_t^{\text{exo}} \right) - \boldsymbol{M}_{t+1} \right\} \\
& + \boldsymbol{\lambda}_{\tau,t}^\top \left\{ \boldsymbol{\sigma} \boldsymbol{\tau}_t + \boldsymbol{e}_1 \sigma^{\text{forc}} \frac{M_{1,t} + G_t}{M_{\text{pre}}} - \boldsymbol{\tau}_{t+1} \right\} \\
& + \boldsymbol{\lambda}_{E,t}^\top \{ \boldsymbol{E}_t \}
\end{aligned}
\tag{5.2}
$$

5.2 Lagrange Conditions

This section outlines the derivation of the Lagrange conditions. Initially, I establish the first-order conditions and use the envelope conditions to eliminate the value function

derivatives from these equations. Finally, these conditions are combined into the conditions that form the policy objective.

5.2.1 First-order Conditions

Let's first consider the condition on costate variables, given in equation (3.8) and proceed with the conditions on the action variables in equation (3.9). The costate variable associated with capital equals the discounted marginal increase in value, resulting from another unit of capital (invested today) in the next period:

$$\lambda_{k,t} = \beta \frac{\partial V(s_{t+1})}{\partial k_{t+1}} \tag{5.3}$$

The condition on the costate variables of the carbon stocks helps in adjusting policy to manage the impacts of carbon reservoir changes, which is needed for estimating the effects of how policies propagate through the climate system:

$$\boldsymbol{\lambda_{M,t}} = \beta \nabla_{\boldsymbol{M_{t+1}}} V(s_{t+1}), \tag{5.4}$$

where $\nabla_{\boldsymbol{M_{t+1}}}$ is the gradient with respect to each of the next-period carbon reservoirs. The costate variable associated with carbon reservoirs is the discounted marginal value of having one more carbon unit in the atmosphere in the next period.

Similarly, the costate variables for the transformed temperatures reflect how changes in the temperature variables at the next time step influence the valuation of the system's state:

$$\boldsymbol{\lambda_{\tau,t}} = \beta \nabla_{\boldsymbol{\tau_{t+1}}} V(s_{t+1}), \tag{5.5}$$

where $\nabla_{\boldsymbol{\tau_{t+1}}}$ is the gradient with respect to each of the next-period transformed temperatures. Each of these conditions relies on a derivative of the value function in the next period. This has to be eliminated so that we can evaluate these expressions. These will be replaced in subsequent subsections.

We now turn our attention to the first-order conditions with respect to the control variables, which will guide the neural network in finding the optimal paths of consumption and emissions within the model. Because the Lagrangian (equation 5.2) attaches a costate variable to each equation of motion, the next-period states are linked to today's controls only through these constraints. A control therefore affects the continuation value $\beta V(\boldsymbol{s}_{t+1})$ exclusively through the costate terms, since $\lambda_{k,t} = \beta \frac{\partial V(s_{t+1})}{\partial k_{t+1}}$ by equation (5.3). Adding a separate term $\beta \frac{\partial V(s_{t+1})}{\partial x_t}$ on top of the costate term would count the same effect twice.

The consumption rate x_t is optimized through the following first-order condition:

$$\frac{1}{x_t} - \lambda_{k,t}\frac{1}{1-x_t} = 0 \tag{5.6}$$

In this equation, $\frac{1}{x_t}$ represents the marginal utility of consumption, indicating that as consumption increases, the additional utility per unit of consumption decreases. The negative term $-\lambda_{k,t}\frac{1}{1-x_t}$ reflects the opportunity cost of consuming more today rather than saving or investing for future consumption. Here, $1 - x_t$ represents the saving rate, and $\lambda_{k,t}$ is the shadow price of capital, which carries the effect of today's consumption on the future value of the system.

The optimization of emissions levels $E_{i,t}$ is governed by this first-order condition:

$$(1 + \lambda_{k,t})\frac{\partial \log F_t(\cdot)}{\partial E_{i,t}} + \lambda_{M_1,t} + \lambda_{E_i,t} = 0 \tag{5.7}$$

The neural network will have to balance the marginal effect of emissions on current production, which enters utility directly through $\frac{\partial \log F_t(\cdot)}{\partial E_{i,t}}$ and capital accumulation through $\lambda_{k,t}\frac{\partial \log F_t(\cdot)}{\partial E_{i,t}}$. Less emissions means less capital. At the same time, emissions also propagate through the climate system. It increases atmospheric CO_2, which has a value of $\lambda_{M_1,t}$. As for the consumption rate, the discounted effect of emissions on the next-period value function is fully captured by the costate variables $\lambda_{k,t}$ and $\lambda_{M_1,t}$ and must not be added separately. Finally, $\lambda_{E_i,t}$ represents the total penalty for any emissions that could drop below zero.

Finally, the last condition is the Karush–Kuhn–Tucker (KKT) condition:

$$E_{i,t} \geq 0 \quad \perp \quad \lambda_{E_i,t} \geq 0, \tag{5.8}$$

where $\perp$ indicates complementary slackness. We must ensure that the loss function is continuously differentiable for backpropagation purposes. Hence, I will use the Fischer-Burmeister function as Maliar et al. (2021) does for complementary slackness:

$$\Psi^{FB}(\lambda_{E_i,t}, E_{i,t}) = \lambda_{E_i,t} + E_{i,t} - \sqrt{\lambda_{E_i,t}^2 + E_{i,t}^2} = 0 \tag{5.9}$$

5.2.2 Envelope Conditions hold in Equilibrium

The derivatives of the value function with respect to the state variables are the marginal effects of the change in choice variables that propagate through the climate system. We need to know these to ensure the neural network makes optimal decisions. We can find

out what they are by using the envelope theorem. The envelope theorem allows us to take the derivative of the value function with respect to the state variables, holding the choice variables constant at their optimal values. We can then replace the value function derivatives in the Lagrange conditions, assuming that the envelope conditions hold when the system is in equilibrium.

More specifically, as we are using a Lagrangian setup, we can use the fact that $\boldsymbol{\lambda}_t$ quantifies the increase in the objective function resulting from a marginal increase in the constraint's right-hand side. As we optimize the controls (put the choices in from the neural network and use them as if they are at their optimal values), we need to know how they affect the value in the next state.

By the envelope conditions, the marginal change in value from one more unit of capital has to equal the direct marginal change in utility plus (the costate) marginal change in the objective function times a marginal change in capital. Capital enters both the utility function (equation 4.19) and the equation of motion for capital (equation 4.20) through $\log F_t(\cdot)$, so both channels must be included:

$$\frac{\partial V(\boldsymbol{s}_t)}{\partial k_t} = \frac{\partial u(x_t)}{\partial k_t} + \lambda_{k,t}\frac{\partial k_{t+1}}{\partial k_t} = (1 + \lambda_{k,t})\frac{\partial \log F_t(\cdot)}{\partial k_t},$$

where I use $F_t(\cdot)$ as shorthand for $F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t)$. This mirrors the envelope condition for the transformed temperatures below, where $\tau_{1,t}$ likewise enters both utility and the capital equation of motion. Iterating this forward $(t + 1)$, we can use it in the capital costate condition (equation 5.3) to eliminate the value function derivative:

$$\lambda_{k,t} = \beta(1 + \lambda_{k,t+1})\frac{\partial \log F_{t+1}(\cdot)}{\partial k_{t+1}} \tag{5.10}$$

We can proceed similarly with the conditions for carbon reservoirs. To be more transparent with calculations, I will first list the equations of the common three-layer carbon model and then summarize it in vector notation, which would hold true for any amount of layer carbon cycle as outlined in the previous section. The conditions we would get with a three-layer carbon cycle are the following:

$$\begin{aligned}\frac{\partial V(\boldsymbol{s}_t)}{\partial M_{1,t}} &= \lambda_{M_1,t}\frac{\partial M_{1,t+1}}{\partial M_{1,t}} + \lambda_{M_2,t}\frac{\partial M_{2,t+1}}{\partial M_{1,t}} + \lambda_{M_3,t}\frac{\partial M_{3,t+1}}{\partial M_{1,t}} + \lambda_{\tau_1,t}\frac{\partial \tau_{1,t+1}}{\partial M_{1,t}} \\ &= \lambda_{M_1,t}\Phi_{1,1} + \lambda_{M_2,t}\Phi_{2,1} + \lambda_{M_3,t}\Phi_{3,1} + \lambda_{\tau_1,t}\sigma^{\text{forc}}\frac{1}{M_{\text{pre}}}\end{aligned}$$

$$\begin{aligned}\frac{\partial V(\boldsymbol{s}_t)}{\partial M_{2,t}} &= \lambda_{M_1,t}\frac{\partial M_{1,t+1}}{\partial M_{2,t}} + \lambda_{M_2,t}\frac{\partial M_{2,t+1}}{\partial M_{2,t}} + \lambda_{M_3,t}\frac{\partial M_{3,t+1}}{\partial M_{2,t}} \\ &= \lambda_{M_1,t}\Phi_{1,2} + \lambda_{M_2,t}\Phi_{2,2} + \lambda_{M_3,t}\Phi_{3,2}\end{aligned}$$

$$\begin{aligned}\frac{\partial V(\boldsymbol{s}_t)}{\partial M_{3,t}} &= \lambda_{M_1,t}\frac{\partial M_{1,t+1}}{\partial M_{3,t}} + \lambda_{M_2,t}\frac{\partial M_{2,t+1}}{\partial M_{3,t}} + \lambda_{M_3,t}\frac{\partial M_{3,t+1}}{\partial M_{3,t}} \\ &= \lambda_{M_1,t}\Phi_{1,3} + \lambda_{M_2,t}\Phi_{2,3} + \lambda_{M_3,t}\Phi_{3,3}\end{aligned}$$

This could be summarized in vector notation as such:

$$\nabla_{M_t} V(\boldsymbol{s}_t) = \boldsymbol{\Phi}^\top \boldsymbol{\lambda_{M,t}} + \boldsymbol{e_1}\lambda_{\tau_1,t}\sigma^{\text{forc}}\frac{1}{M_{\text{pre}}},$$

being valid for any $m \in \mathbb{N}$-layer carbon cycle model according to the specifications of ACE-DICE. We can eliminate the value function derivatives in the Lagrange condition for the carbon reservoirs (equation 5.4) by iterating this one timestep forward and substituting this into it:

$$\boldsymbol{\lambda_{M,t}} = \beta\left(\boldsymbol{\Phi}^\top \boldsymbol{\lambda_{M,t+1}} + \boldsymbol{e_1}\lambda_{\tau_1,t+1}\sigma^{\text{forc}}\frac{1}{M_{\text{pre}}}\right) \tag{5.11}$$

Continuing in the same manner for the envelope conditions on transformed temperatures, I will list the equations in the case of a two-layer temperature model and provide vector notation for the general ACE-DICE envelope conditions. A two-layer temperature model would have the following envelope conditions:

$$\frac{\partial V(\boldsymbol{s}_t)}{\partial \tau_{1,t}} = \lambda_{\tau_1,t}\frac{\partial \tau_{1,t+1}}{\partial \tau_{1,t}} + \lambda_{\tau_2,t}\frac{\partial \tau_{2,t+1}}{\partial \tau_{1,t}} - \xi_0 - \lambda_{k,t}\xi_0 = \lambda_{\tau_1,t}\sigma_{1,1} + \lambda_{\tau_2,t}\sigma_{2,1} - \xi_0 - \lambda_{k,t}\xi_0$$

$$\frac{\partial V(\boldsymbol{s}_t)}{\partial \tau_{2,t}} = \lambda_{\tau_1,t}\frac{\partial \tau_{1,t+1}}{\partial \tau_{2,t}} + \lambda_{\tau_2,t}\frac{\partial \tau_{2,t+1}}{\partial \tau_{2,t}} = \lambda_{\tau_1,t}\sigma_{1,2} + \lambda_{\tau_2,t}\sigma_{2,2}$$

Summarized in vector notation:

$$\nabla_{\tau_t} V(\boldsymbol{s}_t) = \boldsymbol{\sigma}^\top \boldsymbol{\lambda_{\tau,t}} - \boldsymbol{e_1}\xi_0(1 + \lambda_{k,t})$$

This will also be valid for any $l \in \mathbb{N}$-layer temperature model that follows the specifications of the general ACE-DICE. Iterating one step forward and substituting into equation (5.5) yields:

$$\boldsymbol{\lambda_{\tau,t}} = \beta \left(\boldsymbol{\sigma}^\top \boldsymbol{\lambda_{\tau,t+1}} - \boldsymbol{e_1} \xi_0 (1 + \lambda_{k,t+1}) \right), \tag{5.12}$$

5.2.3 Final Objective Functions

This section shows how we end up with the final policy objective. I remind you that the end goal, as discussed in section (3.2), is fitting the equilibrium conditions by regression. For convenience, the general formulation of the policy objective is provided here:

$$\ell_\theta \doteq \frac{1}{b} \sum_{i=1}^{b} \sum_{m=0}^{M} \left[\ell_m \left(\boldsymbol{s}_i, \pi_\theta(\boldsymbol{s}_i) \right) \right]^2. \tag{3.10}$$

The first individual objective functions ℓ_m are the Lagrange conditions, where the costate has to equal the partial derivative of the Lagrange function with respect to the state (equation 3.8), as we have derived them in the previous subsection. The first objective function ℓ_0, will be equation (5.10) rearranged so that it should equal zero at optimum:

$$\ell_0 \doteq \lambda_{k,t} - \beta(1 + \lambda_{k,t+1}) \frac{\partial \log F_{t+1}(\cdot)}{\partial k_{t+1}} \tag{5.13}$$

The other objective functions on states will be the conditions for carbon reservoirs (eq. 5.11) ℓ_1 and transformed temperatures (eq. 5.12) ℓ_2. The two objective functions are:

$$\ell_1 \doteq \boldsymbol{\lambda_{M,t}} - \beta \left(\boldsymbol{\Phi}^\top \boldsymbol{\lambda_{M,t+1}} + \boldsymbol{e_1} \lambda_{\tau_1,t+1} \sigma^{\text{forc}} \frac{1}{M_{\text{pre}}} \right) \tag{5.14}$$

$$\ell_2 \doteq \boldsymbol{\lambda_{\tau,t}} - \beta \left(\boldsymbol{\sigma}^\top \boldsymbol{\lambda_{\tau,t+1}} - \boldsymbol{e_1} \xi_0 (1 + \lambda_{k,t+1}) \right) \tag{5.15}$$

Furthermore, we need the Lagrange conditions for the action variables (eq. 3.9). The condition on the consumption rate (eq. 5.6) will serve as ℓ_3:

$$\ell_3 \doteq \frac{1}{x_t} - \lambda_{k,t} \frac{1}{1 - x_t} \tag{5.16}$$

No further elimination of value function derivatives is needed here, as the effect of x_t on the next-period value is already contained in $\lambda_{k,t}$. One could equivalently write the first-order condition as $\frac{1}{x_t} + \beta \frac{\partial V(s_{t+1})}{\partial k_{t+1}} \frac{\partial k_{t+1}}{\partial x_t} = 0$ and substitute equation (5.3), which yields the same expression. As a check, the analytical solution $x^* = 1 - \beta\kappa$ (equation 4.22) together with the constant costate $\lambda_k^* = \frac{\beta\kappa}{1-\beta\kappa}$ sets both ℓ_0 and ℓ_3 exactly to zero when $\frac{\partial \log F_t(\cdot)}{\partial k_t} = \kappa$.

The first-order conditions on emissions (eq. 5.7) enter the objective function as ℓ_4. For the same reason as for the consumption rate, the effect of emissions on the next-period value is captured by $\lambda_{k,t}$ and $\lambda_{M_1,t}$, which themselves are pinned down by ℓ_0 and ℓ_1:

$$\ell_4 \doteq (1 + \lambda_{k,t}) \frac{\partial \log F_t(\cdot)}{\partial E_{i,t}} + \lambda_{M_1,t} + \lambda_{E_i,t} \tag{5.17}$$

The KKT-condition for non-negative emissions expressed through the Fischer-Burmeister function is objective ℓ_5 and will be given as:

$$\ell_5 \doteq \Psi^{FB}(\lambda_{E_i,t}, E_{i,t}) = \lambda_{E_i,t} + E_{i,t} - \sqrt{\lambda_{E_i,t}^2 + E_{i,t}^2} \tag{5.18}$$

Finally, the Bellman Equation has to hold (where * indicates optimized variables):

$$\ell_6 \doteq \log x_t^* + \log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t^*) - \xi_0 \tau_{1,t} + \xi_0 + \beta V(\boldsymbol{s}_{t+1}) - V(\boldsymbol{s}_t) \tag{5.19}$$

A summary of all the objective functions is provided in Appendix (A).

5.3 Policy Network Input and Outputs

This section details the policy network's inputs and outputs. It starts by highlighting the preprocessing steps necessary to optimize network performance and ensure the inputs are well-suited for the model's requirements. Following the discussion of inputs, I describe the network's outputs, characterizing its elements and dimensions.

5.3.1 Inputs

In general, our policy network receives three types of input: aggregate log capital k_t, $m \in \mathbb{N}$ carbon reservoirs $\boldsymbol{M}_t$, and $l \in \mathbb{N}$ transformed temperature layers $\boldsymbol{\tau}_t$. In addition, I will include time as an exogenous state to account for the non-stationary nature of IAMs, as noted in Folini et al. (2024). Thus, the state space is characterized by the following vector:

$$\boldsymbol{s}_t \in \mathbb{R}^{2+m+l} \doteq \left(k_t, \boldsymbol{M}_t^\top, \boldsymbol{\tau}_t^\top, t\right)^\top \tag{5.20}$$

To maintain computational tractability, some kind of preprocessing of the state variables is necessary. Generally, in deep learning, some form of a normalization procedure is necessary to smooth out differences in magnitude between the features of the input vari-

ables, which can lead to better model convergence. For more on normalization, refer to Huang et al. (2023) and the references therein. More formally, normalization of a dataset $\mathcal{D} = \{\boldsymbol{x}_i\}_{i=1}^N$ is a function $f : \boldsymbol{x} \to \hat{\boldsymbol{x}}$, which results in a transformed dataset $\hat{\mathcal{D}} = \{\hat{\boldsymbol{x}}_i\}_{i=1}^N$ that has certain statistical properties (Huang et al. 2023, Definition II.1.).

There are no explicit normalization practices in the literature specifically related to this thesis, which, as mentioned before, is Folini et al. (2024) and Friedl et al. (2023). Interestingly, normalization is not even common in the broader literature of deep RL, as noted in Huang et al. (2023). However, the two related papers try to reduce the scale of the state variables, which resembles an attempt to perform normalization. Specifically, they scaled capital by productive labor $\frac{K_t}{A_t N_t}$, used 1000 gigaton carbon instead of gigatons, and applied the time transformation in Traeger (2014).

For my thesis, I am using the log of aggregate capital, which already greatly reduces its scale. Additionally, I am following the two mentioned papers in feeding the network 1000 gigatonnes instead of gigatonnes and adopting the time transformation in Traeger (2014), resulting in scaled time $\tilde{t}$ in the interval $(0, 1)$. The time transformation is given as follows:

$$\begin{aligned} \tilde{t} &= 1 - \exp(-\varsigma t), \quad \text{with the inverse} \\ t &= -\frac{\ln(1-\tilde{t})}{\varsigma}. \end{aligned} \tag{5.21}$$

Where the parameter ς is 0.02. With these state normalizations, the preprocessed state vector $\boldsymbol{s}_t^p$ becomes:

$$\boldsymbol{s}_t^p \in \mathbb{R}^{2+m+l} \doteq \left(k_t, \frac{\boldsymbol{M}_t^\top}{1000}, \boldsymbol{\tau}_t^\top, \tilde{t}\right)^\top \tag{5.22}$$

With this, the current state of normalization techniques, particularly for deep learning methods applied to IAMs, as is the case for the present thesis, appears somewhat rudimentary and arbitrarily applied. While functionally adequate in the previous studies, this underscores a gap in the methodological rigor and suggests room for advancement in this area.

5.3.2 Outputs

The outputs of the neural network consist of the consumption rate x_t and I_E resource inputs $\boldsymbol{E}_t$, as well as the vector of costate variables $\boldsymbol{\lambda}_t$. As previously covered, this vector consists of m costate variables for the carbon reservoirs, l costate variables for the temperature layers, and I_E costate variables for the KKT conditions on the resource inputs. The reason why we need to predict the costate variables is because they can't be analytically derived. However, the objective functions allow us to pin them down.

The action vector, representing the outputs of the neural network, is structured as follows:

$$\boldsymbol{a}_t \in \mathbb{R}^{3+I_E+m+l+I_E} \doteq \left(x_t, \boldsymbol{E}_t^\top, V_t(\boldsymbol{s}_t), \lambda_{k,t}, \boldsymbol{\lambda}_{\boldsymbol{M},\boldsymbol{t}}^\top, \boldsymbol{\lambda}_{\tau,t}^\top, \boldsymbol{\lambda}_{E,t}^\top\right)^\top \tag{5.23}$$

In the DICE specification of Appendix (A), the upper bound $E_t \leq E_t^{\text{BAU}}$ adds one more multiplier λ_t^{BAU} to the output vector, so that the policy network has $|\mathcal{A}| = 11$ outputs.

I want to note that Folini et al. (2024) and Friedl et al. (2023), both use k_{t+1} as the control instead of consumption. This approach allows them to circumvent the need to explicitly model the equation of motion for capital in forward simulations. Although this methodological choice simplifies the computational requirements, it should yield equivalent results to the approach I am taking.

6 Results

In this section, I apply DEQNs to specific variations of ACE, namely with the DICE 2016 and 2023 production function as well as using the typical DICE carbon cycles. Because of the project's scope, I want to note that the results in this section are preliminary and that there has not been a focus on calibrating the model, only solving a model with a given set of parameters. The parameters used for the simulations can be found in the appendix (B), and supplementary figures are presented in the appendix (C). First, the section covers the architecture of the policy network used for approximating the equilibrium conditions, the training scheme used to train it, and error statistics in the equilibrium conditions. Then, I will go on to compare the estimates of the consumption rates and abatement rates with the analytical solutions.

6.1 Network Architecture and Hyperparameters

6.1.1 Network

As outlined in section (3.2), the policy network used by DEQNs is a Multilayer Perceptron (MLP), consisting solely of densely connected layers. The specific architecture employed in this thesis follows those of the works by Azinovic et al. (2022), Friedl et al. (2023), and Folini et al. (2024)[10]. Table (2) lists each layer with the respective characteristics, and figure (2) is an illustration of the architecture.

Layer Number	Layer Type	Nodes	Activation Function
1	Input Layer	$\lvert\mathcal{S}\rvert = 7$	None
2	Dense	1024	ReLu
3	Dense	1024	ReLu
4	Output Layer	$\lvert\mathcal{A}\rvert = 11$	Action Specific

Table 2: Network Architecture

The input layer takes the state vector, which in the case of the DICE variations of ACE has a state space that is given by $\boldsymbol{s}_t \in \mathbb{R}^7$. Therefore, the size of the input layer is 7. This is followed by two hidden layers, each comprising 1024 neurons. These layers utilize the ReLU (Rectified Linear Unit) activation function. The output layer gives the action vector, which in the case of the DICE variations of ACE is $\boldsymbol{a}_t \in \mathbb{R}^{11}$, meaning that it has

[10]Folini et al. (2024) does not cover the specific architecture for their policy network in the paper. However, they have a GitHub repository accompanying the paper.

11 units. These outputs are then fed to specific activation functions that map the raw output of the MLP to the appropriate ranges.

Friedl et al. (2023) distinguishes itself from the other papers by using the self-normalizing Scaled Exponential Linear Unit (selu) activation function (Klambauer et al. 2017) in the hidden layers. However, in the interest of simplicity and computational efficiency, I followed the approach of the other two papers and used the relu activation function. The relu function has performed well in previous studies and is easier to compute.

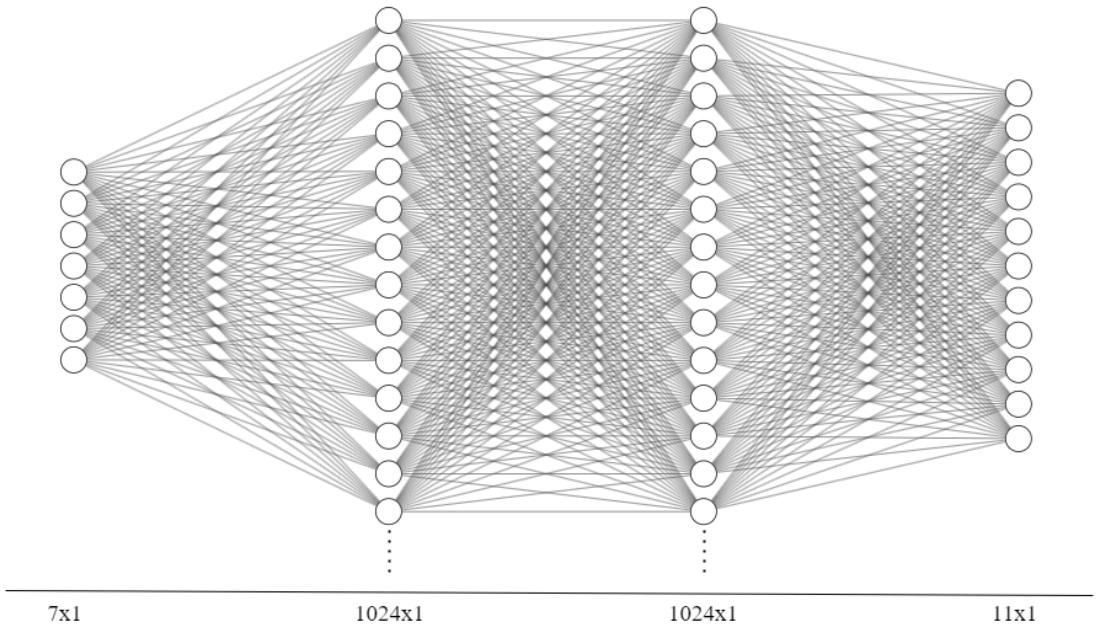


Figure 2: Illustration of policy network architecture

The output layer has specific activation functions tailored to each action variable. The same custom activation functions were utilized in the output layer in both model runs, as detailed in Table (3). The activation functions applied to the costate variables largely mirror those found in Folini et al. (2024), where a linear activation function is applied to each respective costate variable. Each linear output is multiplied by a fixed scale c that matches the order of magnitude of the variable, so that the network itself only has to produce numbers of order one.

To prevent potential issues such as taking the logarithm of zero or dividing by zero, modifications were necessary for the activation functions of the control variables x_t and E_t[11]. The custom sigmoid functions scale inputs between a minimal constant α and an adjusted upper limit b, as depicted in the following equation:

$$f(x) = b\,(\alpha + (1 - 2\alpha)\text{sigmoid}(x)) \tag{6.1}$$

[11]Refer to appendix (A) for the specific functional forms of the two model versions of ACE.

Action Variable	Activation Function	Function Mappings
x_t	Custom Sigmoid	$f : \mathbb{R} \to (\alpha, 1-\alpha)$, α small
E_t	Custom Sigmoid	$f : \mathbb{R} \to (\alpha, E_t^{\mathrm{BAU}}(1-\alpha))$, α small
V_t	Scaled linear, $c_V = 30$	$f : \mathbb{R} \to (-\infty, \infty)$
$\lambda_{k,t}$	Linear, $c_k = 1$	$f : \mathbb{R} \to (-\infty, \infty)$
$\boldsymbol{\lambda}_{M,t}$	Scaled linear, $c_M = 10^{-4}$	$f : \mathbb{R} \to (-\infty, \infty)$
$\boldsymbol{\lambda}_{\tau,t}$	Scaled linear, $c_\tau = 10^{-1}$	$f : \mathbb{R} \to (-\infty, \infty)$
$\lambda_{E,t}, \lambda_t^{\mathrm{BAU}}$	Scaled linear, $c_{\mathrm{KKT}} = 10^{-4}$	$f : \mathbb{R} \to (-\infty, \infty)$

Table 3: Custom Functions

The custom sigmoid function on x_t uses $b = 1$ and the custom sigmoid function on E_t uses $b = E_t^{\mathrm{BAU}}$ in both versions, while land-use emissions E_t^{land} enter the carbon cycle exogenously. The bounded emissions are computed once from the raw network output, and the same value is used in the equations of motion and in every objective function.

6.1.2 Scaling of the Objective Functions

The costate variables differ by several orders of magnitude. At the analytical solution, $\lambda_k \approx 0.35$, while $\lambda_{M_1} \approx -10^{-4}$. The residuals of the conditions involving the carbon costates are correspondingly small, so in an unweighted sum of squares the carbon conditions ℓ_1 and the emissions condition ℓ_4 would barely affect the policy objective, and the network could leave the SCC almost undetermined. I therefore divide each residual by the scale of the costate it pins down: ℓ_1 and ℓ_4 by c_M, and ℓ_2 by c_τ. The Fischer-Burmeister functions are evaluated on the scale-free pairs $(\lambda_{E,t}/c_{\mathrm{KKT}},\, E_t/E_t^{\mathrm{BAU}})$ and $(\lambda_t^{\mathrm{BAU}}/c_{\mathrm{KKT}},\, 1 - E_t/E_t^{\mathrm{BAU}})$. Dividing a residual by a positive constant changes neither its root nor the solution of the system, only how strongly each condition is weighted in the policy objective (eq. 3.10).

6.1.3 Training Scheme

Friedl et al. (2023) implemented a training scheme characterized by a gradual increase in episode length as the policy network stabilized in learning. It started with an episode length of 4 and increased in powers of two, where they, in the final step, went to 300 steps. Furthermore, they used a learning rate of 1e-5 throughout the whole training process. Minibatches were adjusted from 64 to 128 as the network became more stable.

In contrast, the publication by Folini et al. (2024) does not provide explicit details on their training methods. However, analysis of their code indicates a similar learning rate of 1e-5, complemented by gradient clipping at a maximum value of 1.0—a technique not mentioned in Friedl et al. (2023). Their approach also involves increasing episode lengths but not starting as low as in the former study, with lengths set at 50, 100, and 500 steps.

Additionally, the initial training cycles consist of 1,000 iterations, escalating to a final cycle of 10,000 iterations.

Trajectory Length	Mini Batch Size	Epochs	Learning Rate	Gradient Clipping	Training Iterations
128	32	4	1e-3 → 1e-5 (cosine)	1.0 (norm)	3000

Table 4: Training Scheme

Table (4) outlines the training scheme used for the policy network. Both versions of ACE are trained in a single cycle of 3000 iterations. Each iteration simulates one episode of 128 periods (640 years) with the current policy, shuffles the visited states, and performs four epochs of mini-batch gradient descent with mini-batches of 32 states. The Adam optimizer (Kingma et al. 2017) starts with a learning rate of 1e-3 that decays along a cosine schedule to 1e-5, and the global norm of the gradient is clipped at 1.0. After every iteration, the policy objective is evaluated on the full episode, and the weights with the lowest value are kept for the results. On a four-core CPU, one training run takes about 25 minutes.

6.2 Learning Statistics

6.2.1 Training Progress

Figure (3) shows the policy objective (eq. 3.10) over the 3000 training iterations for the DICE 2016 and DICE 2023 versions of ACE. In both versions, the objective falls by several orders of magnitude within the first few iterations, as the network moves from its random initialization to controls and costates of the right magnitude. The initial value is much larger for the 2023 version (about $2 \cdot 10^9$ against 44 for the 2016 version), because the randomly initialized network produces an extreme first episode, but this has no lasting effect on training.

After the initial drop, the objective decreases steadily throughout training, from around 10^{-2} after 300 iterations to $6.3 \cdot 10^{-6}$ for the 2016 version and $4.1 \cdot 10^{-5}$ for the 2023 version in the final iteration. The fluctuations visible in the middle of training shrink as the cosine schedule lowers the learning rate, and neither run shows the large spikes or exploding gradients observed with the original implementation. The lowest objective on a full episode, which determines the weights used for the results, is $6.0 \cdot 10^{-6}$ for the 2016 version and $2.9 \cdot 10^{-5}$ for the 2023 version.

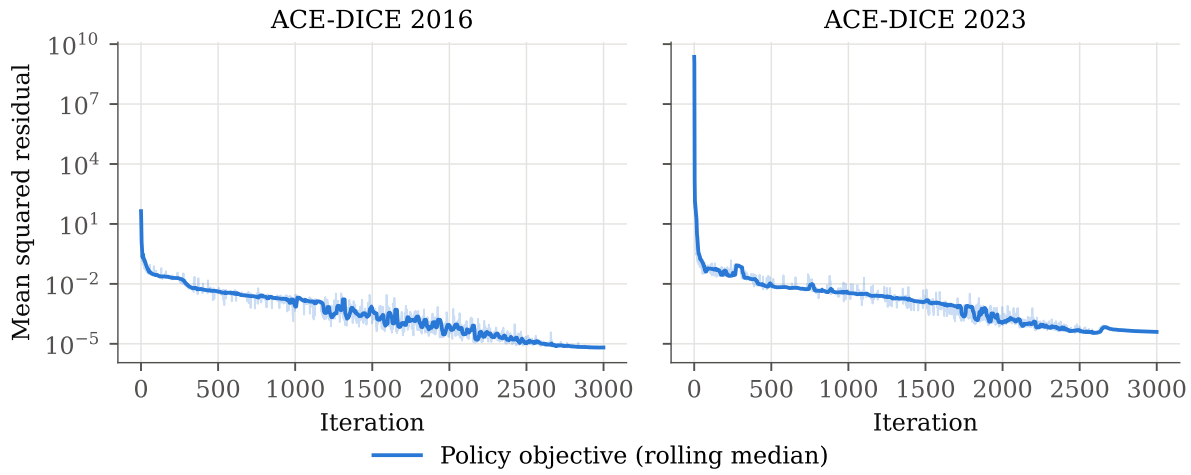


Figure 3: Policy objective over training iterations (light line: per iteration; dark line: rolling median over 25 iterations)

6.2.2 Errors in Equilibrium Conditions

The errors in the individual components that make up the total objective function, as given in equation (3.10), are presented in Tables (5) and (6), and summarized in Figure (4). The statistics in each column are the absolute errors $|\ell_m|$ of the respective condition over an episode of 128 periods simulated with the best-performing network. The residuals are measured in the scaled units of section (6.1.2), so a residual of 10^{-3} in ℓ_1 corresponds to an error of about $10^{-3} \cdot c_M = 10^{-7}$ in the carbon costates, or about 0.1% of their value.

Statistic	ℓ_0	ℓ_{1,M_1}	ℓ_{1,M_2}	ℓ_{1,M_3}	ℓ_{2,τ_1}	ℓ_{2,τ_2}	ℓ_3	ℓ_4	$\ell_{5,A}$	$\ell_{5,B}$	ℓ_6
mean	2.4e-04	3.2e-04	1.4e-04	1.9e-05	2.3e-04	4.6e-05	2.4e-04	7.9e-04	4.4e-04	1.9e-04	7.9e-04
std	2.1e-04	2.6e-04	9.1e-05	1.3e-05	2.0e-04	4.4e-05	2.0e-04	7.8e-04	1.6e-03	1.7e-04	8.0e-04
min	9.8e-07	1.5e-06	2.0e-07	6.5e-07	4.5e-07	3.7e-08	2.5e-06	4.1e-06	9.5e-07	1.2e-06	7.6e-06
25%	8.8e-05	1.3e-04	6.9e-05	9.2e-06	9.4e-05	1.7e-05	1.0e-04	2.8e-04	9.5e-07	5.9e-05	2.3e-04
50%	1.9e-04	2.8e-04	1.4e-04	1.7e-05	1.8e-04	3.8e-05	2.1e-04	5.8e-04	1.1e-06	1.2e-04	5.6e-04
75%	3.4e-04	4.1e-04	2.0e-04	2.8e-05	2.9e-04	5.5e-05	2.9e-04	1.0e-03	1.6e-04	2.9e-04	1.1e-03
99.9%	1.2e-03	1.3e-03	5.1e-04	7.2e-05	1.1e-03	2.8e-04	9.2e-04	4.1e-03	1.4e-02	8.3e-04	4.0e-03
max	1.2e-03	1.3e-03	5.1e-04	7.5e-05	1.1e-03	2.8e-04	9.2e-04	4.1e-03	1.4e-02	8.3e-04	4.0e-03

Table 5: Error Statistics ACE-DICE (2016 DICE), absolute errors $|\ell_m|$

For the 2016 version, the mean absolute errors of all conditions lie between $2 \cdot 10^{-5}$ and $8 \cdot 10^{-4}$, and the maximum error over the episode is below $5 \cdot 10^{-3}$ for every condition except the Fischer-Burmeister function $\ell_{5,A}$. The largest errors are in the emissions condition ℓ_4 and the Bellman equation ℓ_6. The errors for the 2023 version are mostly larger, by up to a factor of seven, but the maximum error remains below 10^{-2} for every condition except $\ell_{5,A}$. For the consumption condition ℓ_3 and the Bellman equation ℓ_6, which are measured in the same units as in the original implementation, the mean errors are about two orders of magnitude smaller than before. The comparison is only indicative, as the original errors measured residuals of different, partly incorrect, conditions.

Statistic	ℓ_0	ℓ_{1,M_1}	ℓ_{1,M_2}	ℓ_{1,M_3}	ℓ_{2,τ_1}	ℓ_{2,τ_2}	ℓ_3	ℓ_4	$\ell_{5,A}$	$\ell_{5,B}$	ℓ_6
mean	1.7e-03	1.3e-03	4.4e-04	1.5e-05	8.1e-04	1.4e-04	1.4e-03	1.2e-03	8.6e-04	4.9e-04	2.3e-03
std	1.4e-03	8.9e-04	3.5e-04	1.3e-05	6.6e-04	1.3e-04	1.1e-03	1.4e-03	2.8e-03	3.8e-04	1.9e-03
min	6.9e-06	3.9e-05	3.5e-06	4.0e-08	3.4e-06	1.5e-06	1.9e-05	1.3e-05	9.5e-07	1.6e-06	7.6e-06
25%	6.0e-04	6.3e-04	2.0e-04	5.9e-06	1.9e-04	5.2e-05	6.0e-04	3.7e-04	9.5e-07	1.7e-04	8.8e-04
50%	1.3e-03	1.1e-03	3.1e-04	1.2e-05	6.7e-04	9.4e-05	1.1e-03	7.0e-04	1.1e-06	3.9e-04	1.6e-03
75%	2.6e-03	1.8e-03	6.2e-04	2.2e-05	1.3e-03	1.6e-04	2.0e-03	1.7e-03	3.5e-04	8.1e-04	3.3e-03
99.9%	5.8e-03	4.1e-03	1.7e-03	7.0e-05	2.8e-03	5.1e-04	5.2e-03	9.0e-03	2.3e-02	1.6e-03	8.2e-03
max	5.8e-03	4.1e-03	1.7e-03	7.1e-05	2.8e-03	5.1e-04	5.3e-03	9.3e-03	2.4e-02	1.7e-03	8.2e-03

Table 6: Error Statistics ACE-DICE (2023 DICE), absolute errors $|\ell_m|$

The largest errors of $\ell_{5,A}$ occur at the few periods in which the analytical abatement rate reaches one and the constraint $E_t \geq 0$ starts to bind, around 2175 for the 2016 version and 2190 for the 2023 version. There, the complementarity condition switches between its two branches, which the smooth policy network can only approximate, and the median error of $\ell_{5,A}$ is about 10^{-6} in both versions.

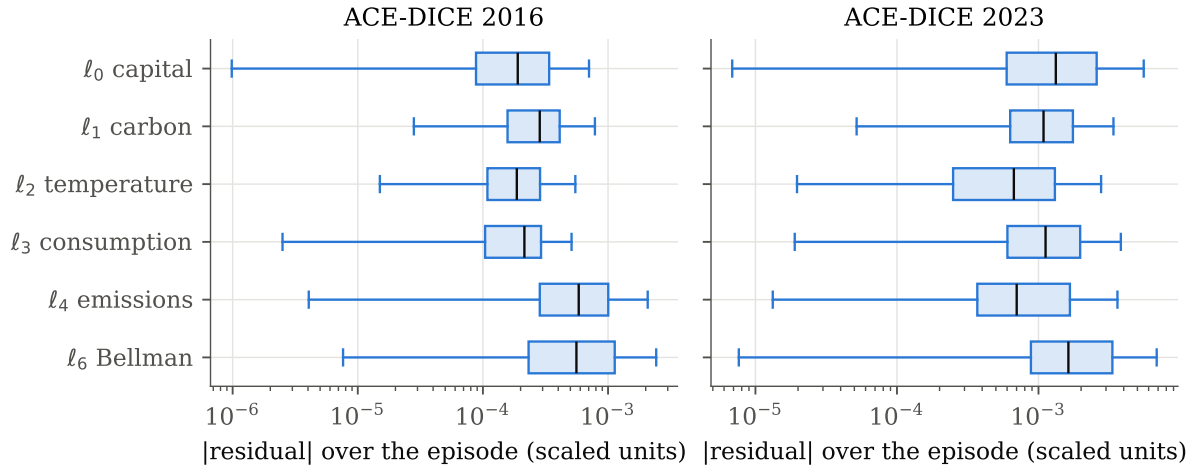


Figure 4: Distribution of the absolute errors over the simulated episode. For conditions with several components, the largest component in each period is shown. Boxes show the interquartile range, whiskers the full range.

6.3 Analytical Comparisons

This section compares the simulated results with the analytical solution of ACE. I begin with the consumption rate and the abatement rate, continue with the costate variables and the SCC they imply, and end with the simulated paths of the states. Throughout, the figures show the paths up to the year 2300, and the policy network is evaluated along its own simulated trajectory. Table (7) summarizes the main comparisons.

	ACE-DICE 2016		ACE-DICE 2023	
	DEQN	Analytic	DEQN	Analytic
Consumption rate x (mean to 2300)	0.7394	0.7395	0.7396	0.7395
Costate λ_k	0.3524	0.3524	0.3520	0.3524
Costate λ_{M_1} ($\times 10^{-4}$)	-1.046	-1.046	-1.044	-1.046
Costate λ_{τ_1}	-0.0559	-0.0559	-0.0559	-0.0559
SCC in 2050 (USD/tCO_2)	36.9	37.0	37.3	37.4
Temperature in 2100 (°C)	2.96	2.96	3.47	3.47
Peak temperature (°C)	3.33	3.33	4.71	4.72
Abatement rate, mean $\lvert\mu - \mu^*\rvert$	0.0009		0.0020	
Abatement rate, max $\lvert\mu - \mu^*\rvert$	0.0166		0.0227	

Table 7: Policy network (DEQN) against the analytical solution. Costates are averaged over the periods up to 2300; the abatement rate errors are taken over the same periods.

6.3.1 The Consumption Rate

As shown in equation (4.22), the optimal consumption rate is constant over time and independent of the states visited, $x^* = 1 - \beta\kappa \approx 0.7395$. The upper panels of Figure (5) show the consumption rates estimated by the policy network, together with the analytical rate and the consumption rates estimated with the original implementation.

The estimated consumption rates match the analytical solution closely. Over the periods up to 2300, the largest deviation from x^* is $6 \cdot 10^{-4}$ for the 2016 version and $2.8 \cdot 10^{-3}$ for the 2023 version, and the policy network has learned that the consumption rate does not depend on the state. In contrast, the original implementation estimated a consumption rate that rises from about 0.53 to 0.75 for the 2016 version and lies above 0.89 for the 2023 version. These deviations were not caused by the normalization of inputs, as suggested in the submitted version of this thesis, but by the errors in the capital condition ℓ_0 and the consumption condition ℓ_3: at the analytical solution, the original conditions are not zero, so the network was trained towards a different fixed point.

6.3.2 The Abatement Rate

Given the carbon price implied by the costate variables, the optimal abatement rate solves the first-order condition on emissions (eq. 5.17), which is equivalent to equation (4.23) with $\Gamma_t = SCC_t$. Where the solution exceeds one, the constraint $E_t \geq 0$ binds and the abatement rate is one.

The lower panels of Figure (5) show that the estimated abatement rates follow the analytical rates closely, from about 9% in the first period to full abatement in 2175 for the 2016 version and in 2195 for the 2023 version. The mean absolute deviation over the

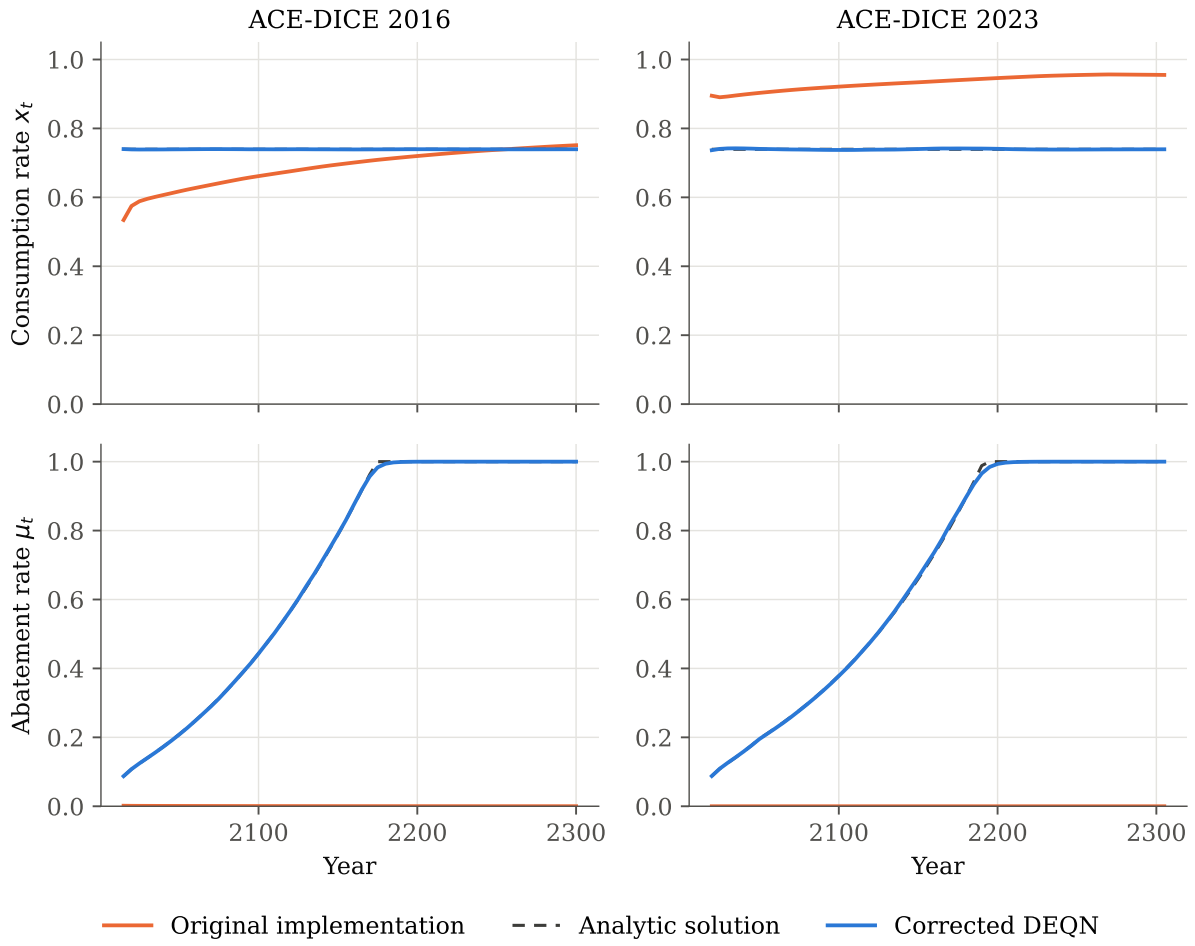


Figure 5: Consumption rate (top) and abatement rate (bottom): corrected DEQN, analytical solution and original implementation

periods up to 2300 is 0.0009 for the 2016 version and 0.0020 for the 2023 version. The largest deviations, of about 0.02, occur in the few periods around the kink where the abatement rate reaches one, which the smooth policy network rounds off. The original implementation estimated abatement rates close to zero in both versions.

6.3.3 Costate Variables and the Social Cost of Carbon

The costate variables of ACE are constant along the optimal path (Appendix C). The policy network recovers them with a largest relative deviation over the periods up to 2300 of 0.3% for λ_k, 0.5% for λ_{M_1} and 0.4% for λ_{τ_1} in the 2016 version, and of 1.4%, 1.4% and 0.5% in the 2023 version. This is notable for the carbon costate $\lambda_{M_1} \approx -1.05 \cdot 10^{-4}$, which is four orders of magnitude smaller than λ_k and is only identified through the conditions on the carbon cycle and on emissions. Without the scaling of section (6.1.2), these conditions carry almost no weight in the policy objective.

Figure (6) shows the SCC implied by the costates of the policy network, as given below

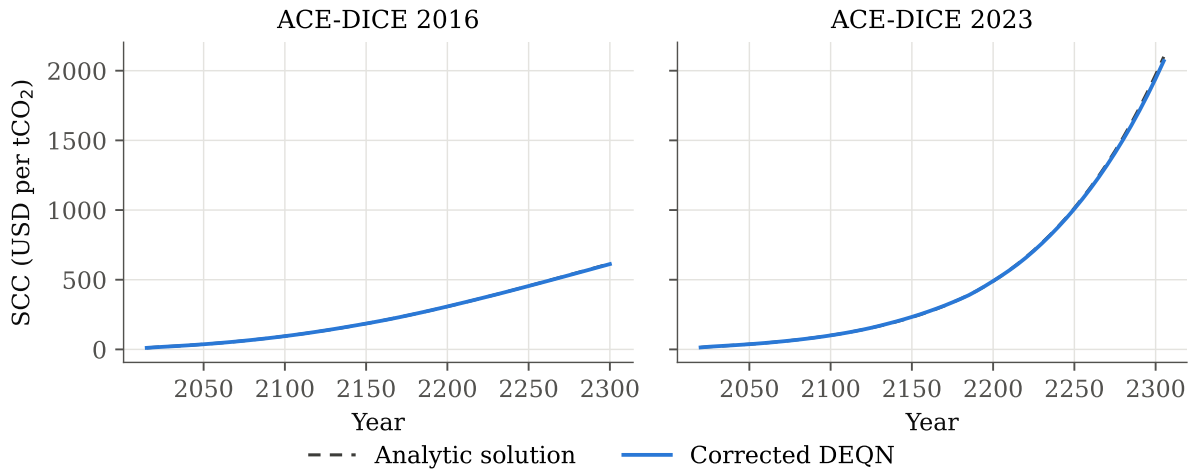


Figure 6: SCC in USD/tCO_2 implied by the policy network and the analytical solution

equation (4.24), together with the analytical SCC (eq. 4.24) evaluated at the same states. The two are almost indistinguishable. For the 2016 version, the SCC rises from 11.1 USD/tCO_2 in 2015 (analytical: 11.0) to 95.1 USD/tCO_2 in 2100 (95.1). For the 2023 version, it rises from 14.1 USD/tCO_2 in 2020 (14.2) to 100.5 USD/tCO_2 in 2100 (100.9). The SCC is proportional to net output, so it grows faster in the 2023 version, where total factor productivity grows faster.

6.3.4 Simulated Paths

Figure (7) compares the states along the trajectory simulated with the policy network to the optimal path obtained by simulating the analytical controls from the same initial state. Since the estimated controls are close to optimal, the two paths coincide. In the 2016 version, atmospheric temperature peaks at 3.33°C around 2150 and atmospheric carbon at about 1590 GtC. In the 2023 version, where emissions are higher because of faster output growth, temperature peaks at 4.71°C around 2185 (analytical: 4.72°C). The small differences after 2200 in the 2023 version reflect the deviations in the abatement rate around the kink.

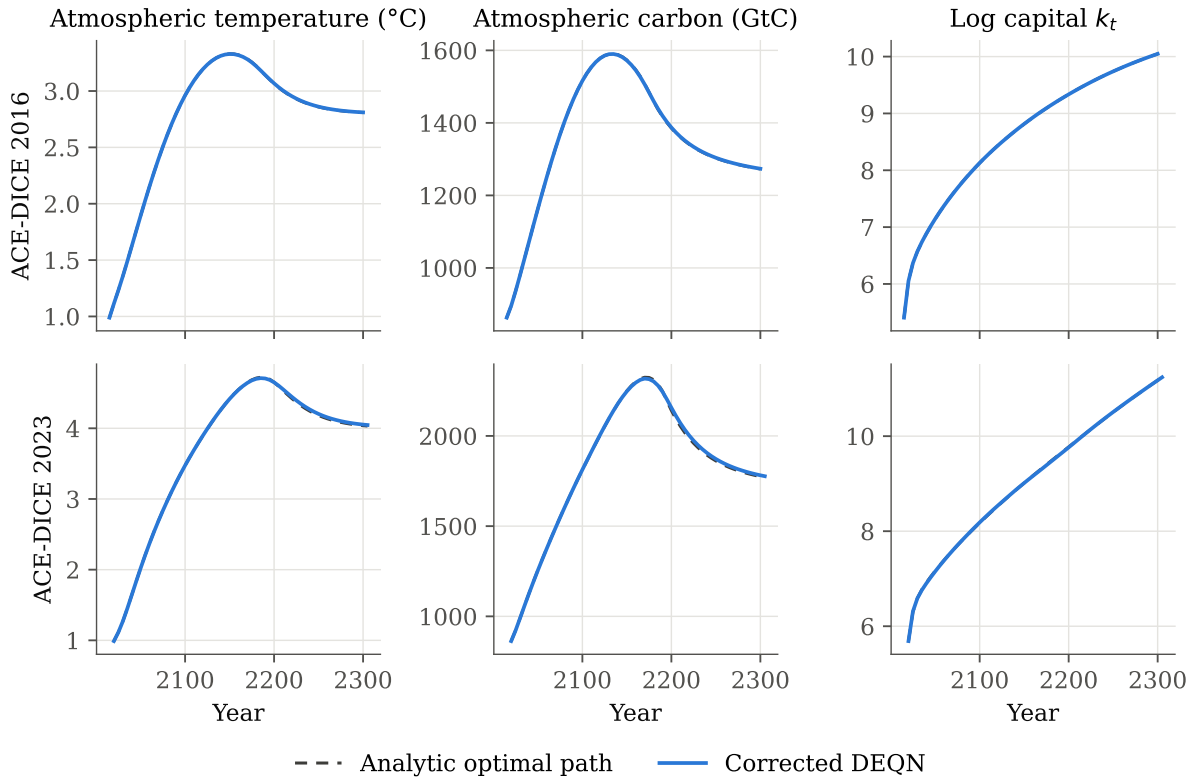


Figure 7: Atmospheric temperature, atmospheric carbon and log capital along the trajectory simulated with the policy network and along the analytical optimal path

7 Conclusion

This thesis marks one of the first applications of DEQNs to an IAM. It details the methodology required to adapt ACE to the DEQN framework, offering a more comprehensive treatment than previous studies. Given ACE's general structure, the methods and derivations in this thesis should be easily extended to other IAMs, facilitating broader adoption in future research.

The thesis conducts two experiments utilizing the mapping of ACE to the DEQN framework, with the DICE 2016 and DICE 2023 production functions. With the corrected equilibrium conditions, the policy network recovers the analytical solution closely in both experiments. The estimated consumption rate stays within 0.3 percentage points of the constant analytical rate, the estimated abatement rate follows the analytical path to full abatement with a mean deviation of 0.1 to 0.2 percentage points, and the costate variables, including the carbon costate that is four orders of magnitude smaller than the capital costate, are recovered to within 1.5%. As a result, the SCC implied by the policy network and the simulated paths of temperature and atmospheric carbon are close to their analytical counterparts.

The revision of this thesis also carries a methodological lesson. In the submitted version, the envelope condition for capital omitted the direct effect of capital on utility, and the first-order conditions counted the effect of today's controls on the future value twice. The policy network then converged towards a wrong fixed point, and I attributed the poor results to the normalization of inputs. Because ACE has an analytical solution, the conditions can be verified directly by evaluating them at the known optimum, which reveals such errors before any network is trained. For models without a closed-form solution, checking the equilibrium conditions on a simplified version with a known solution is therefore a valuable step. A second lesson is that the conditions should be weighted so that each carries comparable weight in the policy objective: without scaling, the conditions on the carbon cycle are several orders of magnitude smaller than the others and barely identify the SCC.

This work establishes a connection with RL by comprehensively discussing concepts common to both DEQNs and the broader RL literature. Contrary to previous research that classified this algorithm as unsupervised learning, this thesis argues that the DEQN algorithm should be categorized as model-based RL. This reclassification provides a foundation for the potential integration of more sophisticated RL algorithms in future economic research. With a correctly specified objective, natural next steps are to relax the assumptions that make ACE analytically tractable, such as the treatment of business-as-usual

emissions as given and the logarithmic utility, and to introduce uncertainty, where the advantages of DEQNs over traditional solution methods become relevant.

References

Abraham, John et al. (2022). "The ocean response to climate change guides both adaptation and mitigation efforts". eng. In: *Atmospheric and oceanic science letters = Daqi-he-haiyang-kexue-kuaibao* 15.4, p. 100221. ISSN: 1674-2834.

Achiam, Joshua (2018). "Spinning Up in Deep Reinforcement Learning". In.

Azinovic, Marlon, Luca Gaegauf, and Simon Scheidegger (2022). "DEEP EQUILIBRIUM NETS". eng. In: *International economic review (Philadelphia)* 63.4, pp. 1471–1525. ISSN: 0020-6598.

Becker, Sebastian, Patrick Cheridito, and Arnulf Jentzen (2020). *Deep optimal stopping*. arXiv: 1804.05394 [math.NA].

Bellman, Richard (1957). "A Markovian Decision Process". eng. In: *Indiana University mathematics journal* 6.5, pp. 679–684. ISSN: 0095-9057.

— (1961). *Adaptive control processes : a guided tour*. eng. Princeton, N.J.

Bosetti, Valentina (Jan. 2021). *Integrated Assessment Models for Climate Change*. DOI: 10.1093/acrefore/9780190625979.013.572. URL: https://oxfordre.com/economics/view/10.1093/acrefore/9780190625979.001.0001/acrefore-9780190625979-e-572.

Brumm, Johannes and Simon Scheidegger (2017). "Using Adaptive Sparse Grids to Solve High-Dimensional Dynamic Models". eng. In: *Econometrica* 85.5, pp. 1575–1612. ISSN: 0012-9682.

Chollet, François et al. (2015). *Keras*. https://keras.io.

Dasgupta, Partha and Geoffrey Heal (1974). "The Optimal Depletion of Exhaustible Resources". eng. In: *The Review of economic studies* 41.5, pp. 3–28. ISSN: 0034-6527.

Duarte, Victor, Diogo Duarte, and Dejanir Silva (Apr. 2018). *Machine Learning for Continuous-Time Finance*. Available at SSRN. DOI: 10.2139/ssrn.3012602. URL: https://ssrn.com/abstract=3012602.

Fernández-Villaverde, Jesús (2016). *Solution and Estimation Methods for DSGE Models*. eng. Cambridge, Mass.

Fernández-Villaverde, Jesús, Samuel Hurtado, and Galo Nuño (2023). "Financial Frictions and the Wealth Distribution". eng. In: *Econometrica* 91.3, pp. 869–901. ISSN: 0012-9682.

Folini, Doris et al. (2024). "The Climate in Climate Economics". eng. In: *The Review of economic studies*. ISSN: 0034-6527.

Friedl, Aleksandra et al. (2023). "Deep uncertainty quantification: with an application to integrated assessment models". Working paper.

Gerlagh, Reyer and Matti Liski (2018). "Consistent Climate Policies". eng. In: *Journal of the European Economic Association* 16.1, pp. 1–44. ISSN: 1542-4766.

Golosov, Mikhail et al. (2014). "Optimal taxes on fossil fuel in general equilibrium". eng. In: *Econometrica* 82.1, pp. 41–88. ISSN: 0012-9682.

Goodfellow, Ian, Yoshua Bengio, and Aaron Courville (2016). *Deep Learning*. `http://www.deeplearningbook.org`. MIT Press.

Grohs, Philipp et al. (2023). "A proof that artificial neural networks overcome the curse of dimensionality in the numerical approximation of Black-Scholes partial differential equations". eng. In: *arXiv.org*. ISSN: 2331-8422.

Gu, Chengyang, Hui Xiong, and Yize Chen (2024). *Pontryagin Optimal Control via Neural Networks*. arXiv: `2212.14566 [eess.SY]`.

Hambel, Christoph, Holger Kraft, and Eduardo Schwartz (2021). "The social cost of carbon in a non-cooperative world". eng. In: *Journal of international economics* 131, p. 103490. ISSN: 0022-1996.

Heal, Geoffrey (1984). "Interactions between Economy and Climate: A Framework for Policy Design under Uncertainty". In: *Advances in Applied Microeconomics*. Ed. by V. Kerry Smith and Ann Dryden White. Vol. 3. Greenwich, CT: JAI Press, pp. 151–168.

Heitzig, Jobst, Wolfram Barfuss, and Jonathan Donges (2018). "A Thought Experiment on Sustainable Management of the Earth System". eng. In: *Sustainability (Basel, Switzerland)* 10.6, p. 1947. ISSN: 2071-1050.

Hoel, Michael and Larry Karp (2002). "Taxes versus quotas for a stock pollutant". eng. In: *Resource and energy economics* 24.4, pp. 367–384. ISSN: 0928-7655.

Huang, Lei et al. (2023). "Normalization Techniques in Training DNNs: Methodology, Analysis and Application". eng. In: *IEEE transactions on pattern analysis and machine intelligence* 45.8, pp. 10173–10196. ISSN: 0162-8828.

Interagency Working Group on the Social Cost of Carbon (2013). *Technical Support Document: Technical Update of the Social Cost of Carbon for Regulatory Impact Analysis under Executive Order 12866*. Technical Report. Washington, DC: United States Government.

Jentzen, Arnulf, Diyora Salimova, and Timo Welti (2019). "A proof that deep artificial neural networks overcome the curse of dimensionality in the numerical approximation of Kolmogorov partial differential equations with constant diffusion and nonlinear drift coefficients". eng. In: *arXiv.org*. ISSN: 2331-8422.

Joos, F et al. (2013). "Carbon dioxide and climate impulse response functions for the computation of greenhouse gas metrics: a multi-model analysis". eng. In: *Atmospheric chemistry and physics* 13.5, pp. 2793–2825. ISSN: 1680-7324.

Judd, Kenneth L., Lilia Maliar, and Serguei Maliar (2011). "Numerically stable and accurate stochastic simulation approaches for solving dynamic economic models: Ap-

proaches for solving dynamic models". eng. In: *Quantitative economics* 2.2, pp. 173–210. ISSN: 1759-7323.

Karp, Larry (2017). "PROVISION OF A PUBLIC GOOD WITH MULTIPLE DYNASTIES". eng. In: *The Economic journal (London)* 127.607, pp. 2641–2664. ISSN: 0013-0133.

Karp, Larry and Jiangfeng Zhang (2012). "Taxes versus quantities for a stock pollutant with endogenous abatement costs and asymmetric information". eng. In: *Economic theory* 49.2, pp. 371–409. ISSN: 0938-2259.

Kingma, Diederik P. and Jimmy Ba (2017). *Adam: A Method for Stochastic Optimization*. arXiv: 1412.6980 [cs.LG].

Klambauer, Günter et al. (2017). *Self-Normalizing Neural Networks*. arXiv: 1706.02515 [cs.LG].

Krueger, Dirk and Felix Kubler (2004). "Computing equilibrium in OLG models with stochastic production". eng. In: *Journal of economic dynamics & control* 28.7, pp. 1411–1436. ISSN: 0165-1889.

Lee, H. and J. Romero, eds. (2023). *Climate Change 2023: Synthesis Report. Contribution of Working Groups I, II and III to the Sixth Assessment Report of the Intergovernmental Panel on Climate Change*. [Core Writing Team, H. Lee and J. Romero (eds.)] Geneva, Switzerland: IPCC, pp. 35–115. DOI: 10.59327/IPCC/AR6-9789291691647.

Maliar, Lilia and Serguei Maliar (2014). "Numerical Methods for Large-Scale Dynamic Economic Models". eng. In: *Handbook of Computational Economics*. Vol. 3. The Netherlands: Elsevier B.V, pp. 325–477. ISBN: 9780444529800.

— (2015). "Merging simulation and projection approaches to solve high-dimensional problems with an application to a new Keynesian model". eng. In: *Quantitative economics* 6.1, pp. 1–47. ISSN: 1759-7331.

Maliar, Lilia, Serguei Maliar, and Pablo Winant (2021). "Deep learning for solving dynamic economic models". eng. In: *Journal of monetary economics* 122, pp. 76–101. ISSN: 0304-3932.

Metcalf, Gilbert E. and James H. Stock (2017). "Integrated Assessment Models and the Social Cost of Carbon: A Review and Assessment of U.S. Experience". eng. In: *Review of environmental economics and policy* 11.1, pp. 80–99. ISSN: 1750-6816.

Mnih, Volodymyr et al. (2013). "Playing Atari with Deep Reinforcement Learning". eng. In: *arXiv (Cornell University)*. ISSN: 2331-8422.

Nitzbon, Jan, Jobst Heitzig, and Ulrich Parlitz (2017). "Sustainability, collapse and oscillations in a simple World-Earth model". eng. In: *Environmental research letters* 12.7, p. 74020. ISSN: 1748-9326.

Nordhaus, William (2011). *Estimates of the Social Cost of Carbon : Background and Results from the RICE-2011 Model*. eng. Cambridge, Mass.

Nordhaus, William (2018). “Projections and Uncertainties about Climate Change in an Era of Minimal Climate Policies”. eng. In: *American economic journal. Economic policy* 10.3, pp. 333–360. ISSN: 1945-7731.

Nordhaus, William and Lint Barrage (2023). *Policies, Projections, and the Social Cost of Carbon : Results from the DICE-2023 Model*. eng. Cambridge, Mass.

Nordhaus, William D (1979). *The efficient use of energy resources*. eng. New Haven.

Pizer, W. A (1999). “The optimal choice of climate change policy in the presence of uncertainty: Climate economics and policy”. eng. In: *Resource and energy economics* 21.3-4, pp. 255–287. ISSN: 0928-7655.

Schauer, Michael (1995). “Estimation of the greenhouse gas externality with uncertainty”. eng. In: *Environmental & resource economics* 5.1, pp. 71–82. ISSN: 0924-6460.

Scheidegger, Simon and Ilias Bilionis (2019). “Machine learning for high-dimensional dynamic stochastic economies”. eng. In: *Journal of computational science* 33, pp. 68–82. ISSN: 1877-7503.

Shakya, Ashish Kumar, Gopinatha Pillai, and Sohom Chakrabarty (2023). “Reinforcement learning algorithms: A brief survey”. eng. In: *Expert systems with applications* 231, p. 120495. ISSN: 0957-4174.

Smolyak, Sergei Abramovich (1963). “Quadrature and interpolation formulas for tensor products of certain classes of functions”. In: *Doklady Akademii Nauk*. Vol. 148. 5. Russian Academy of Sciences, pp. 1042–1045.

Strnad, Felix M. et al. (Dec. 2019). “Deep reinforcement learning in World-Earth system models to discover sustainable management strategies”. In: *Chaos: An Interdisciplinary Journal of Nonlinear Science* 29.12, p. 123122. ISSN: 1054-1500. DOI: 10.1063/1.5124673. eprint: https://pubs.aip.org/aip/cha/article-pdf/doi/10.1063/1.5124673/14624621/123122_1_online.pdf. URL: https://doi.org/10.1063/1.5124673.

Sutton, Richard S. (2018). *Reinforcement learning : an introduction*. eng. Cambridge, Massachusetts.

Taylor, Karl E., Ronald J. Stouffer, and Gerald A. Meehl (2012). “AN OVERVIEW OF CMIP5 AND THE EXPERIMENT DESIGN”. eng. In: *Bulletin of the American Meteorological Society* 93.4, pp. 485–498. ISSN: 0003-0007.

Traeger, Christian P. (2014). “A 4-Stated DICE: Quantitatively Addressing Uncertainty Effects in Climate Change”. eng. In: *Environmental & resource economics* 59.1, pp. 1–37. ISSN: 0924-6460.

— (2021). “IAMs and CO2 Emissions-An Analytic Discussion”. In: *EAERE Conference*.

— (2023). “ACE—Analytic Climate Economy”. eng ; nor. In: *American economic journal. Economic policy* 15.3, pp. 372–406. ISSN: 1945-7731.

Valaitis, Vytautas and Alessandro T. Villa (2024). “A machine learning projection method for macro-finance models”. eng. In: *Quantitative economics* 15.1, pp. 145–173. ISSN: 1759-7323.

Winschel, Viktor and Markus Krätzig (2010). “Solving, Estimating, and Selecting Nonlinear Dynamic Models Without the Curse of Dimensionality”. eng. In: *Econometrica* 78.2, pp. 803–821. ISSN: 0012-9682.

Wolf, Theodore et al. (2023). “Can Reinforcement Learning support policy makers? A preliminary study with Integrated Assessment Models”. eng. In: *arXiv.org*. ISSN: 2331-8422.

A DEQN Objective Details

A.1 General ACE: Policy Objective Summary

$$\ell_0 \doteq \lambda_{k,t} - \beta(1+\lambda_{k,t+1})\frac{\partial \log F_{t+1}(\cdot)}{\partial k_{t+1}} \tag{5.13}$$

$$\ell_1 \doteq \boldsymbol{\lambda_{M,t}} - \beta\left(\boldsymbol{\Phi}^\top \boldsymbol{\lambda_{M,t+1}} + \boldsymbol{e_1}\lambda_{\tau_1,t+1}\sigma^{\text{forc}}\frac{1}{M_{\text{pre}}}\right) \tag{5.14}$$

$$\ell_2 \doteq \boldsymbol{\lambda_{\tau,t}} - \beta\left(\boldsymbol{\sigma}^\top \boldsymbol{\lambda_{\tau,t+1}} - \boldsymbol{e_1}\xi_0(1+\lambda_{k,t+1})\right) \tag{5.15}$$

$$\ell_3 \doteq \frac{1}{x_t} - \lambda_{k,t}\frac{1}{1-x_t} \tag{5.16}$$

$$\ell_4 \doteq (1+\lambda_{k,t})\frac{\partial \log F_t(\cdot)}{\partial E_{i,t}} + \lambda_{M_1,t} + \lambda_{E_i,t} \tag{5.17}$$

$$\ell_5 \doteq \lambda_{E_i,t} + E_{i,t} - \sqrt{\lambda_{E_i,t}^2 + E_{i,t}^2} \tag{5.18}$$

$$\ell_6 \doteq \log x_t^* + \log F(\boldsymbol{A}_t, \boldsymbol{N}_t, \boldsymbol{K}_t, \boldsymbol{E}_t^*) - \xi_0\tau_{1,t} + \xi_0 + \beta V(\boldsymbol{s}_{t+1}) - V(\boldsymbol{s}_t) \tag{5.19}$$

A.2 Model-specific Mapping

For the derivatives, we will need the log of the production function. I will rewrite the log of the following variables $\log A_t = a_t, \log K_t = k_t$. The log of the production function is:

$$\begin{aligned}\log Y_t &= \log F_t(\cdot) \\ &= a_t + \kappa k_t + (1-\kappa)\log N_t + \log\left[1 - \theta_{1,t}\left(1 - \frac{E_t}{E_t^{\text{BAU}}}\right)^{\theta_2}\right]\end{aligned} \tag{A.1}$$

where E_t^{BAU} is treated as an exogenous variable. In DICE, business-as-usual emissions are proportional to gross output, $E_t^{\text{BAU}} = \sigma_t Y_t^{\text{gross}}$, so they depend on capital. The planner, however, takes the sequence E_t^{BAU} as given when choosing controls, and the simulation evaluates E_t^{BAU} at the realized capital stock. This keeps $\log F_t(\cdot)$ linear in k_t, which is what makes the analytical solution of section (4.3) available as a benchmark. We need the derivative of $\log Y_t$ with respect to k_t:

$$\frac{\partial \log F_t(\cdot)}{\partial k_t} = \kappa$$

We also need the derivative of Y_t w.r.t. E_t:

$$\begin{aligned}\frac{\partial \log F_t(\cdot)}{\partial E_t} &= \left(1-\theta_{1,t}\left(1-\frac{E_t}{E_t^{\text{BAU}}}\right)^{\theta_2}\right)^{-1}\left(-\theta_{1,t}\theta_2\left(1-\frac{E_t}{E_t^{\text{BAU}}}\right)^{\theta_2-1}\right)\left(-\frac{1}{E_t^{\text{BAU}}}\right)\\ &= \frac{\theta_{1,t}\theta_2\left(1-\frac{E_t}{E_t^{\text{BAU}}}\right)^{\theta_2-1}}{E_t^{\text{BAU}}\left(1-\theta_{1,t}\left(1-\frac{E_t}{E_t^{\text{BAU}}}\right)^{\theta_2}\right)}\end{aligned}$$

Finally, we need to make sure that we are choosing E_t such that $0 \leq E_t \leq E_t^{\text{BAU}}$. This gives an additional constraint and KKT condition:

$$E_t^{\text{BAU}} - E_t \geq 0 \quad \perp \quad \lambda_t^{\text{BAU}} \geq 0 \tag{A.2}$$

along with the Fischer-Burmeister function, where the multiplier λ_t^{BAU} enters the first-order condition on emissions with a negative sign:

$$\Psi^{\text{FB}}(\lambda_t^{\text{BAU}}, E_t^{\text{BAU}} - E_t) = \lambda_t^{\text{BAU}} + (E_t^{\text{BAU}} - E_t) - \sqrt{(\lambda_t^{\text{BAU}})^2 + (E_t^{\text{BAU}} - E_t)^2} \tag{A.3}$$

In the DICE calibration, emissions E_t are measured in GtCO_2 per year, while the carbon reservoirs $\boldsymbol{M}_t$ are measured in GtC and the model is solved with a time step of Δ years. The atmospheric carbon stock therefore increases by $\frac{\partial M_{1,t+1}}{\partial E_t} = \zeta \equiv \frac{\Delta}{3.664}$ per unit of yearly emissions, where 3.664 converts CO_2 to carbon, and this factor scales $\lambda_{M_1,t}$ in the first-order condition on emissions. Here, I restate the objective functions with the DICE production function:

$$\ell_0 \doteq \beta(1+\lambda_{k,t+1})\kappa - \lambda_{k,t} \tag{A.4}$$

$$\ell_1 \doteq \beta\left(\boldsymbol{\Phi}^\top \boldsymbol{\lambda_{M,t+1}} + \boldsymbol{e_1}\lambda_{\tau_1,t+1}\sigma^{\text{forc}}\frac{1}{M_{\text{pre}}}\right) - \boldsymbol{\lambda_{M,t}} \tag{A.5}$$

$$\ell_2 \doteq \beta\left(\boldsymbol{\sigma}^\top \boldsymbol{\lambda_{\tau,t+1}} - \boldsymbol{e_1}\xi_0(1+\lambda_{k,t+1})\right) - \boldsymbol{\lambda_{\tau,t}} \tag{A.6}$$

$$\ell_3 \doteq \frac{1}{x_t} - \frac{\lambda_{k,t}}{1-x_t} \tag{A.7}$$

$$\ell_4 \doteq \frac{\partial \log F_t(\cdot)}{\partial E_t}(1+\lambda_{k,t}) + \zeta\lambda_{M_1,t} + \lambda_{E,t} - \lambda_t^{\text{BAU}} \tag{A.8}$$

$$\ell_{5,A} \doteq \lambda_{E,t} + E_t - \sqrt{\lambda_{E,t}^2 + (E_t)^2} \tag{A.9}$$

$$\ell_{5,B} \doteq \lambda_t^{\text{BAU}} + (E_t^{\text{BAU}} - E_t) - \sqrt{(\lambda_t^{\text{BAU}})^2 + (E_t^{\text{BAU}} - E_t)^2} \tag{A.10}$$

$$\ell_6 \doteq \log x_t^* + a_t + \kappa k_t + (1-\kappa)\log N_t + \log\left[1 - \theta_{1,t}\left(1 - \frac{E_t^*}{E_t^{\text{BAU}}}\right)^{\theta_2}\right] \tag{A.11}$$
$$- \xi_0\tau_{1,t} + \xi_0 + \beta V(\boldsymbol{s}_{t+1}) - V(\boldsymbol{s}_t)$$

B Simulation Initializations

B.1 Exogenous variables

The law of motion for labor is given by:

$$N_{t+1} = N_t \left(\frac{N_\infty}{N_t} \right)^{\delta^N} \tag{B.1}$$

Parameter	Symbol	Value DICE 2016	Value DICE 2023
World pop. (Millions)	N_0	7403	7752.9
Asymptotic world pop (Millions)	N_∞	11500	10825.0
Convergence rate	δ^N	0.134	0.145

Table 8: Labor Evolution Parameterization

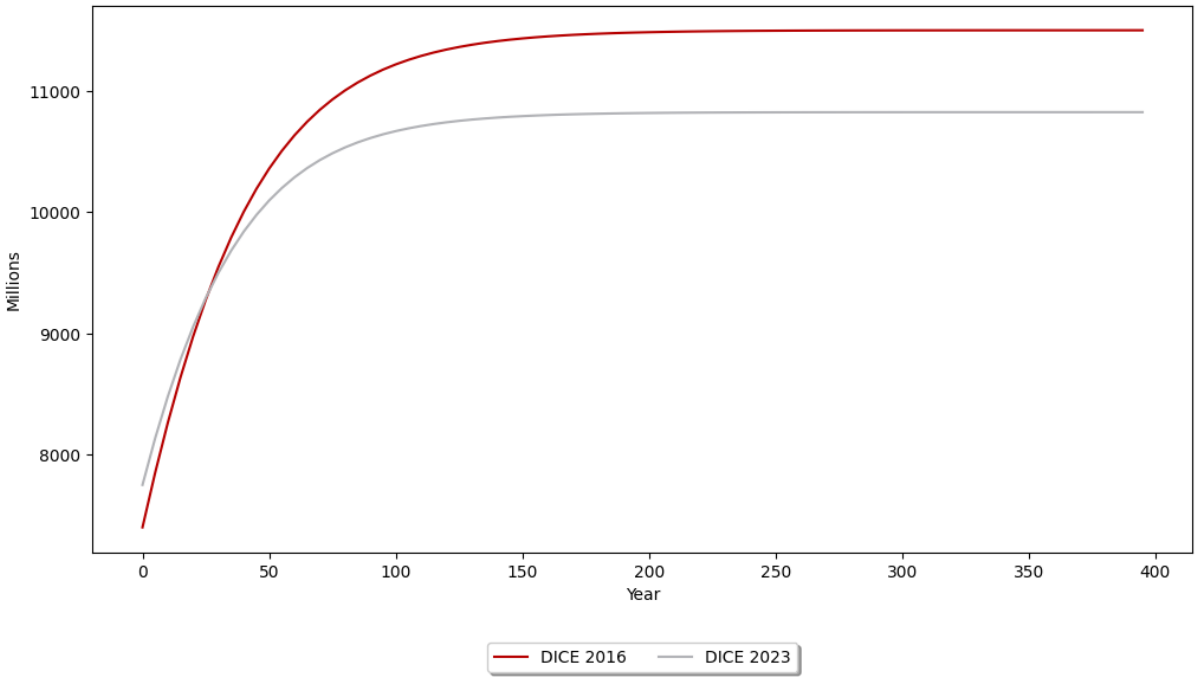


Figure 8: Exogenous law of motion for labor

Total factor productivity evolves according to:

$$A_{t+1} = \frac{A_t}{1 - g_t^A}, \quad \text{with} \tag{B.2}$$

$$g_{t+1}^{A} = g_0^{A} \exp\left(-\delta^{A} \Delta (t-1)\right) \tag{B.3}$$

Parameter	Symbol	Value DICE 2016	Value DICE 2023
Initial TFP	A_0	5.115	5.84
Initial TFP Growth (pr. 5yrs)	g_0^A	0.075	0.066
Decline Rate of TFP Growth	δ^A	0.005	0.015
Time Step	Δ	5	5

Table 9: TFP Evolution Parameterization

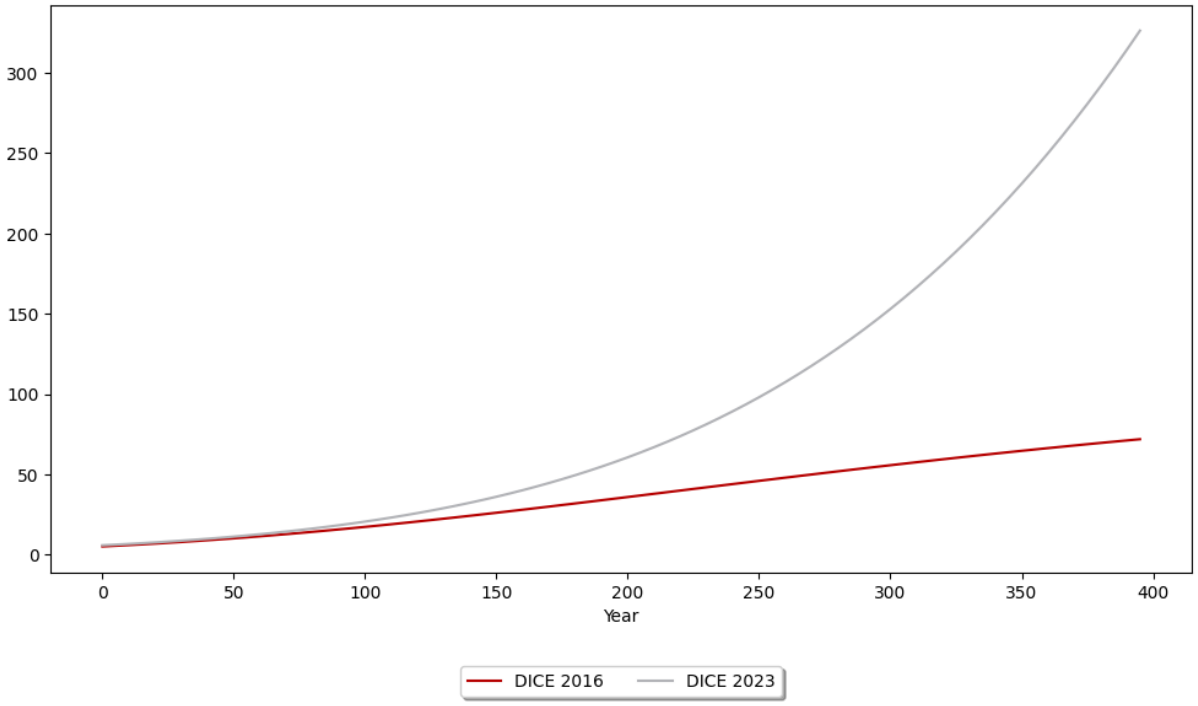


Figure 9: Exogenous law of motion for tfp

Carbon intensity relates anthropogenic carbon emissions to economic production and is a fraction of total output. DICE 2016 uses:

$$\sigma_{t+1} = \exp(g_t^{\sigma} \Delta) \tag{B.4}$$

$$g_{t+1}^{\sigma} = g_t^{\sigma} \left(1 + \delta^{\sigma}\right)^{\Delta} \tag{B.5}$$

While DICE 2023 uses:

$$\sigma_{t+1} = \sigma_t \exp\left(\Delta g_t^{\sigma}\right) \mathfrak{e}_{t+1} \tag{B.6}$$

$$g_t^\sigma = \min(g_0^\sigma(\delta^\sigma)^{t-1}, g_\infty^\sigma) \tag{B.7}$$

The current ratio of CO_2e to industrial CO_2 is:

$$\mathfrak{e}_t = \begin{cases} \mathfrak{e}_{2020} + \frac{\mathfrak{e}_{2100} - \mathfrak{e}_{2020}}{16}(t-1), & \text{if } t \leq 16 \\ \mathfrak{e}_{2100} - \mathfrak{e}_{2020}, & \text{if } t \geq 17 \end{cases} \tag{B.8}$$

And both using:

$$\sigma_0 = \frac{E_0}{Y_0(1-\mu_0)} \tag{B.9}$$

Parameter	Symbol	Value DICE 2016	Value DICE 2023
Initial growth of sigma (pr. yr.)	g_0^σ	-0.0152	-0.015
Decline rate of decarbonization	δ^σ	-0.001	0.96
Initial Industrial Emissions ($GtCO_2$)	E_0	35.85	37.56
Initial World Gross Output (trillion$)	Y_0	105.5	135.7
Initial Abatement Rate	μ_0	0.03	0.05
Time Step	Δ	5	5
Ratio of CO_2e to industrial CO_2 2020	$\mathfrak{e}_{2020}$	-	1.40
Ratio of CO_2e to industrial CO_2 2100	$\mathfrak{e}_{2100}$	-	1.21

Table 10: Carbon Intensity Evolution Parameterization

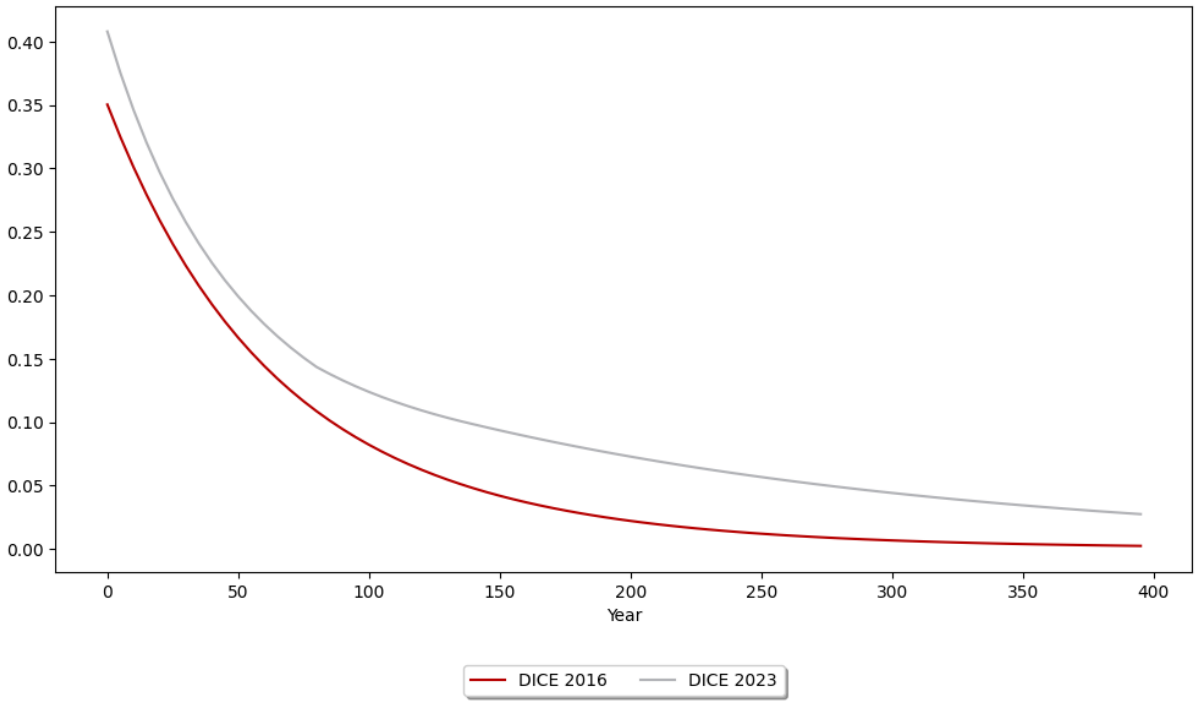


Figure 10: Exogenous law of motion for carbon intensity

Abatement costs depend on this backstop price that is calculated differently between the two model versions. The 2016 version is given as:

$$p_t^{\text{back}} = \tilde{p}(1 - g^{\text{back}})^{t-1} \tag{B.10}$$

The 2023 version is as follows:

$$p_t^{\text{back}} = \begin{cases} \bar{p} \times \exp(-\Delta \times 0.01 \times (t-7)), & \text{if } t \leq 7 \\ \bar{p} \times \exp(-\Delta \times 0.001 \times (t-7)), & \text{if } t > 7 \end{cases} \tag{B.11}$$

Abatement costs in both versions are given by:

$$\theta_{1,t} = \frac{p_t^{\text{back}} \sigma_t}{1000\theta_2} \tag{B.12}$$

Parameter	Symbol	Value DICE 2016	Value DICE 2023
Initial growth of sigma (pr. yr.)	g_0^σ	-0.0152	-0.015
Decline rate of decarbonization	δ^σ	-0.001	0.96
Initial Industrial Emissions ($GtCO_2$)	E_0	35.85	37.56
Initial World Gross Output (trillion$)	Y_0	105.5	135.7
Initial Abatement Rate	μ_0	0.03	0.05
Time Step	Δ	5	5
Ratio of CO_2e to industrial CO_2 2020	$\mathfrak{e}_{2020}$	-	1.40
Ratio of CO_2e to industrial CO_2 2100	$\mathfrak{e}_{2100}$	-	1.21

Table 11: Abatement Cost Evolution Parameterization

Emissions from deforestation in the 2016 version are given by:

$$E_t^{\text{land}} = E_0^{\text{land}}(1 - \delta^{\text{land}})^{t-1} \tag{B.13}$$

Parameter	Symbol	Value DICE 2016	Value DICE 2023
Initial Land Use Emissions	E_0^{land}	2.6	5.9
Land Use Decline Rate (yr.)	δ^{forest}	0.115	0.1

Table 12: Land Use Emissions Evolution Parameterization

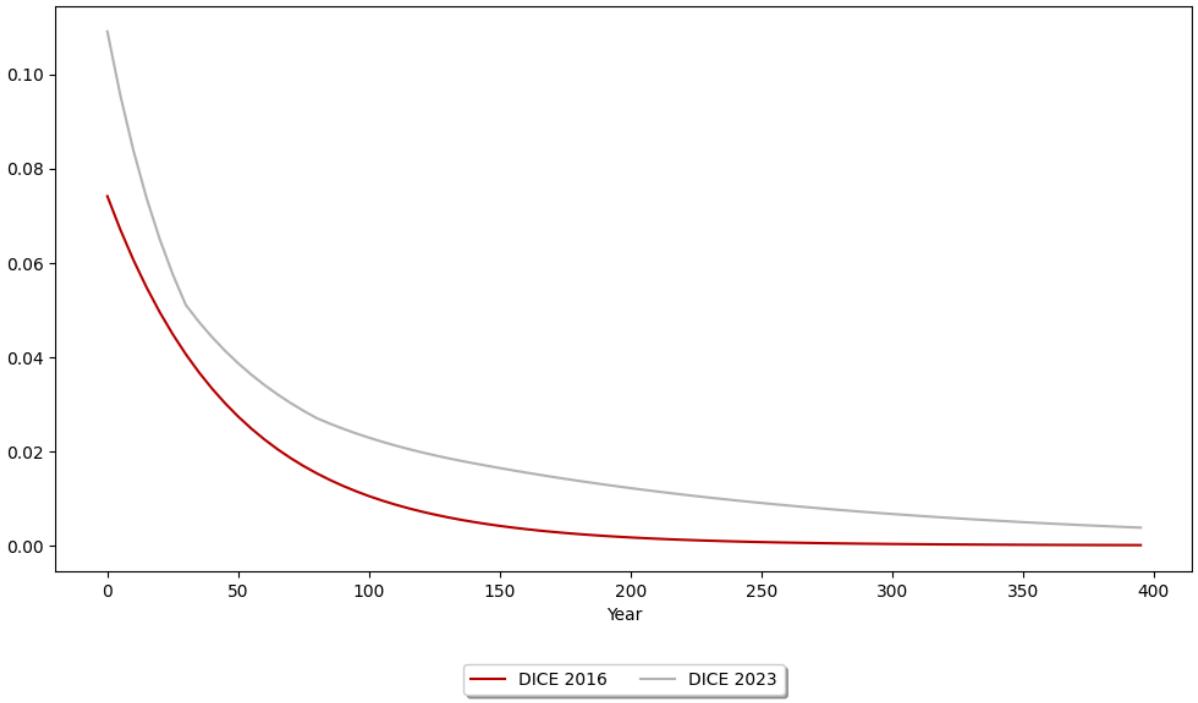


Figure 11: Exogenous law of motion for abatement costs

B.2 Endogenous Variables

Economic parameterization. The discount rate is initialized as:

$$\beta = \left(\frac{1}{1+\rho}\right)^{\Delta} \tag{B.14}$$

Calibrated Parameter	Symbol	Value	Model
Pure Rate of Time Preference	ρ	0.0286	ACE
Capital Share of Output	κ	0.3	ACE
Capital Depreciation Factor	δ_k	0.0439	ACE
Growth Rate of Capital	g_k	0.02949	ACE
Time Step	Δ	5	DICE (All)
Initial Capital Stock 2015 (trillion 2010$)	K_0	223	DICE 2016
Initial Capital Stock 2020 (trillion 2019$)	K_0	295	DICE 2023
Damage parameter	ξ_0	0.02219	ACE
Parameter	ξ_1	$\frac{\log 2}{s} = 0.2310,\ s = 3$	ACE

Table 13: Generic Parameterization of the Economy

Carbon parameterization. The vector that initializes DICE's 5-year step carbon cycle's transition matrix is given in Traeger (2023) and is

$$\Xi = \begin{bmatrix} 0.088 & 0.0025 & 0.03832888889 & 0.00033750 \end{bmatrix} \tag{B.15}$$

The components of the transition matrix are constructed as:

$$\Phi = \begin{bmatrix} 1-\Xi_1 & \Xi_3 & 0 \\ \Xi_1 & 1-\Xi_2 & \Xi_4 \\ 0 & \Xi_2 & 1-\Xi_4 \end{bmatrix} \tag{B.16}$$

Parameter	Symbol	ACE
Atmospheric Carbon Stock 2015 (GtC)	$M_{1,0}$	862.86
Upper Ocean Carbon Stock 2015 (GtC)	$M_{2,0}$	1541.11
Deep Ocean Carbon Stock 2015 (GtC)	$M_{3,0}$	10010.44

Table 14: Generic Parameterization of the Carbon Reservoirs

Temperature parameterization.

Parameter	Symbol	Value ACE
Temperature Layer 1	$\tau_{1,0}$	~ 1.2567272
Temperature Layer 2	$\tau_{2,0}$	~ 1.1854651
Heat to space	σ_1^{up}	0.24755
Heat from second to first layer	σ_2^{up}	0.011354
Heat from first to second layer	σ_1^{down}	0.1479

Table 15: Generic Parameterization of Temperatures

The transition matrix for the two-layer temperature model is given by:

$$\begin{bmatrix} 1-\sigma_1^{\text{up}}-\sigma_1^{\text{down}} & \sigma_1^{\text{down}} \\ \sigma_2^{\text{up}} & 1-\sigma_2^{\text{up}} \end{bmatrix} \tag{B.17}$$

Furthermore, σ^{forc} is given by:

$$\sigma^{\text{forc}} = 1 - \boldsymbol{\sigma}_{1,1} - \boldsymbol{\sigma}_{1,2} \tag{B.18}$$

C Supplementary Figures

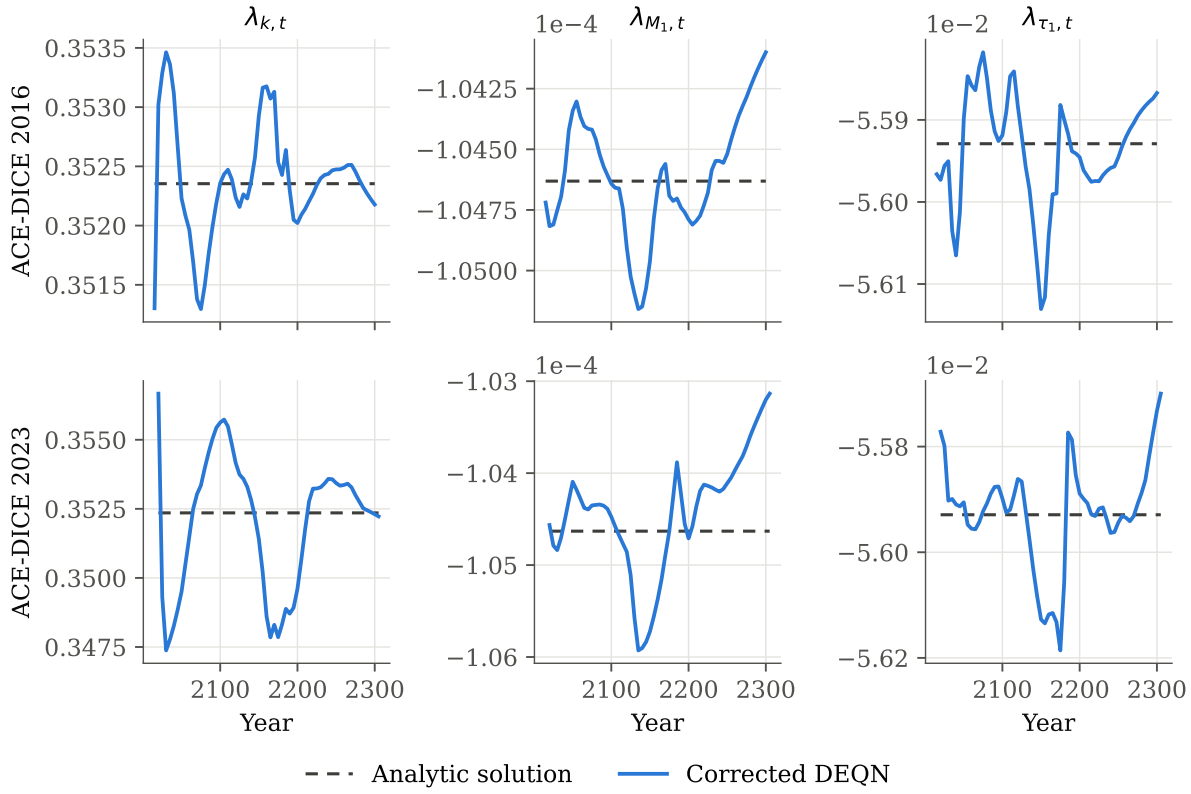


Figure 12: Costate variables $\lambda_{k,t}$, $\lambda_{M_1,t}$ and $\lambda_{\tau_1,t}$ estimated by the policy network along its simulated trajectory, against their constant analytical values. Note the narrow vertical ranges: the deviations are below 1.5% in all panels.